Deep Learning 1

EECS 492

Introduction to Artificial Intelligence

Credits: Many slides from Prof. Jia-Bin Huang, Prof. Fei Fei Li, Prof. Christopher Manning and Prof. Dan Jurafsky

Reminders

- HW1 due tomorrow!
- This week's topic for discussion section: deep learning

Reminders

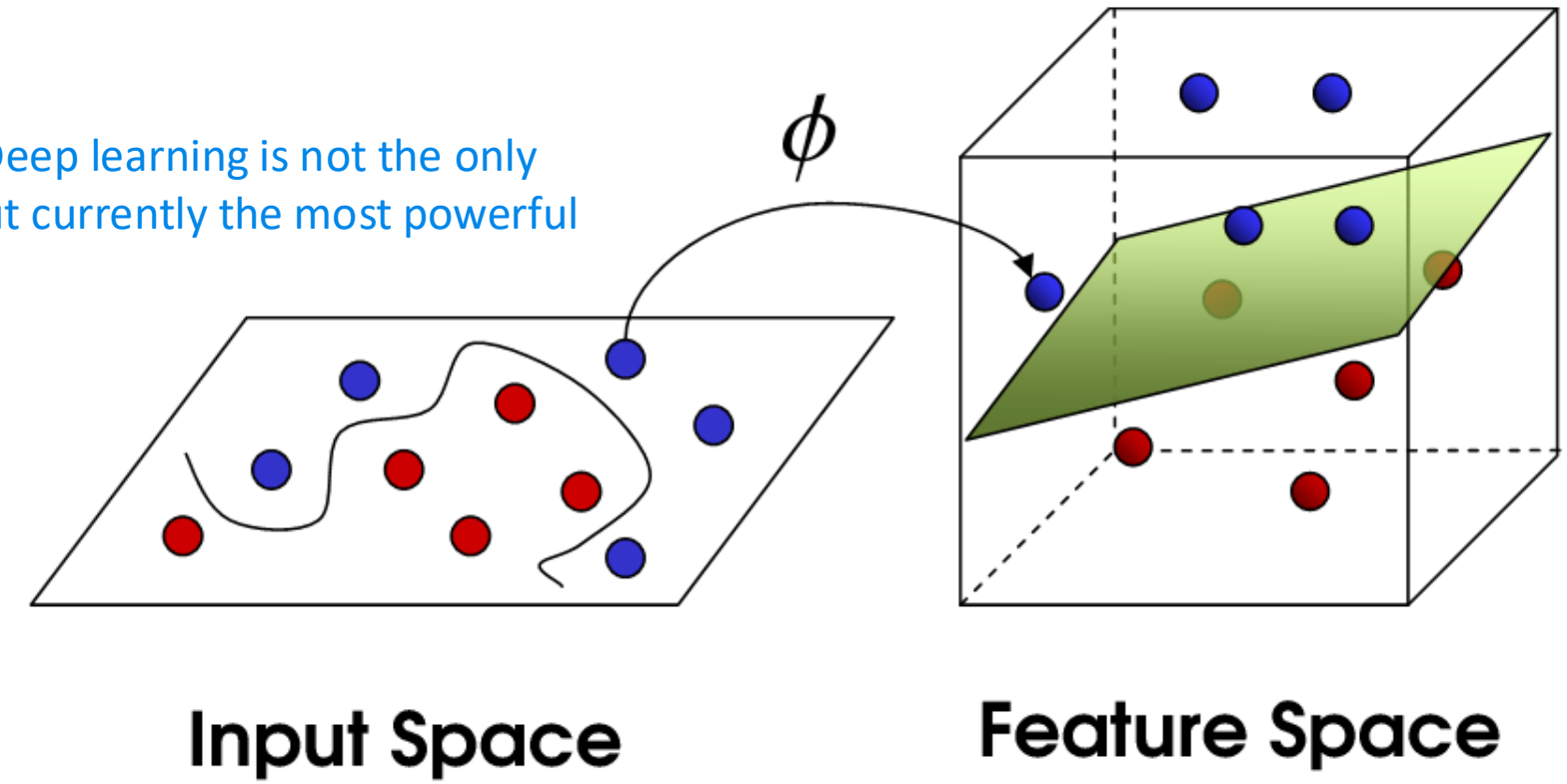
- Ethics discussion on February 13
  - Complete reading quiz before the session
  - Make-up: come to professor office hours the following week (come at the beginning of office hours)
    - 1-1:30 PM on M-W-TH

Today's Class

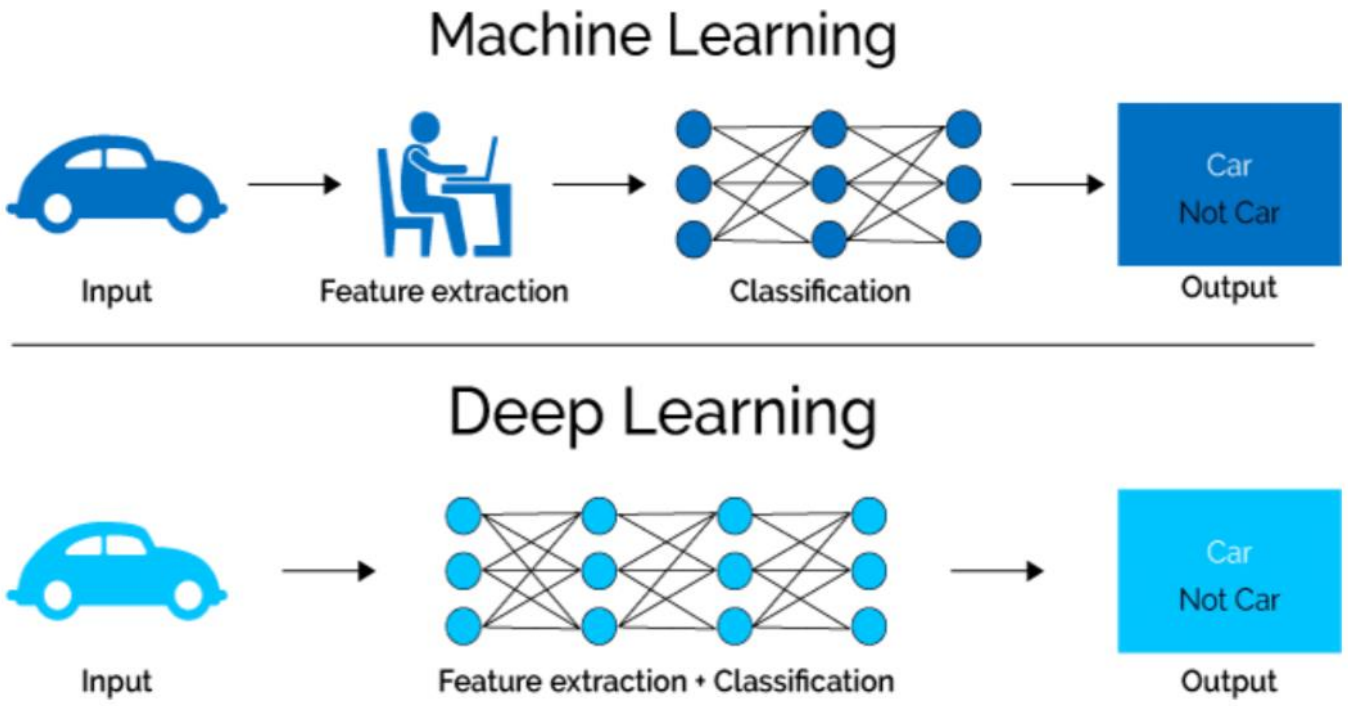
- Neural Networks
- Convolutional Neural Nets (CNNs)

Neural Networks Enable Automated Feature Engineering

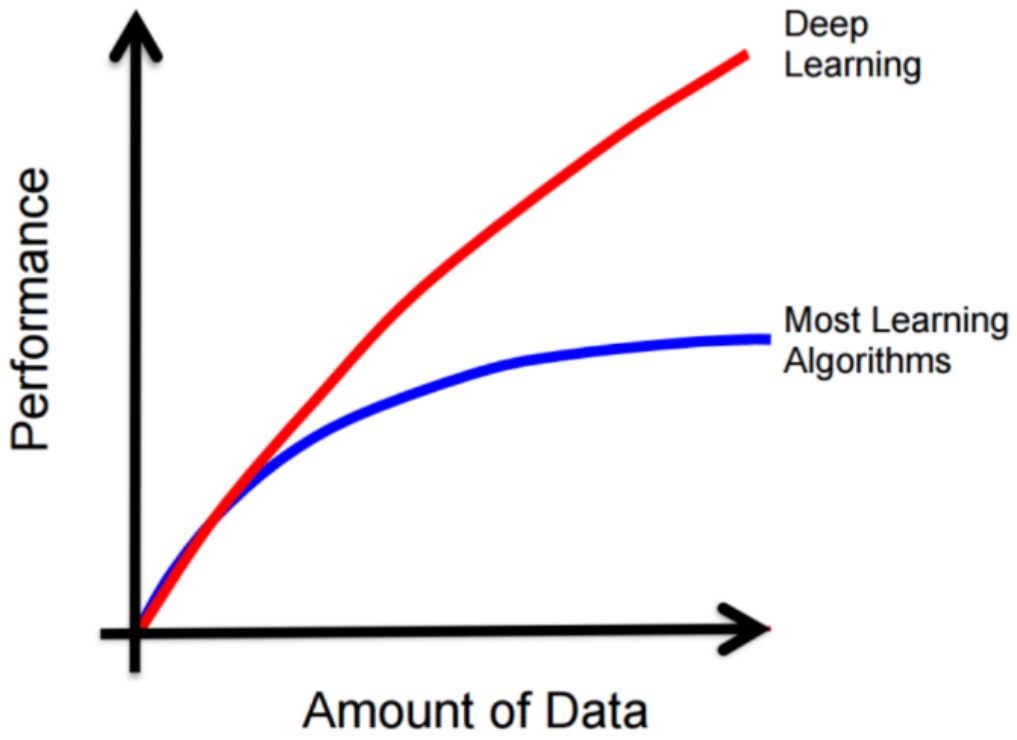
Note: Deep learning is not the only one, but currently the most powerful



Deep Learning Automates the Creation of Knowledge Representations



https://deeplearning.mit.edu/



Neural Network Unit

- Take weighted sum of inputs, plus a bias

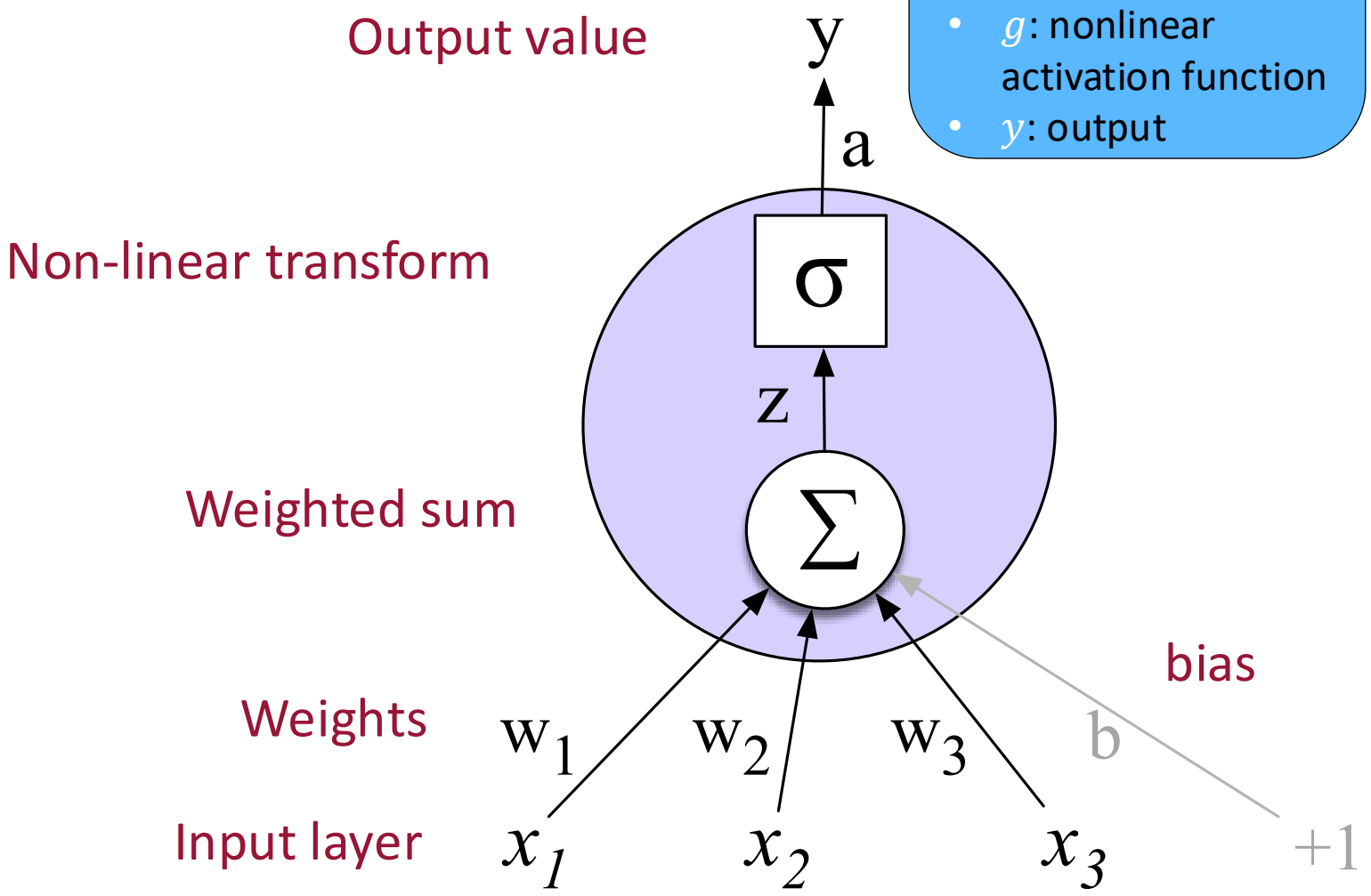
$$z = b + \sum_i w_i x_i$$

$$z = w \cdot x + b$$

- Instead of just using z, we'll apply a nonlinear activation function g:

$$y = a = g(z)$$

- x: input vector
- w: weight vector
- b: bias term
- z: output before nonlinear function
- g: nonlinear activation function
- y: output

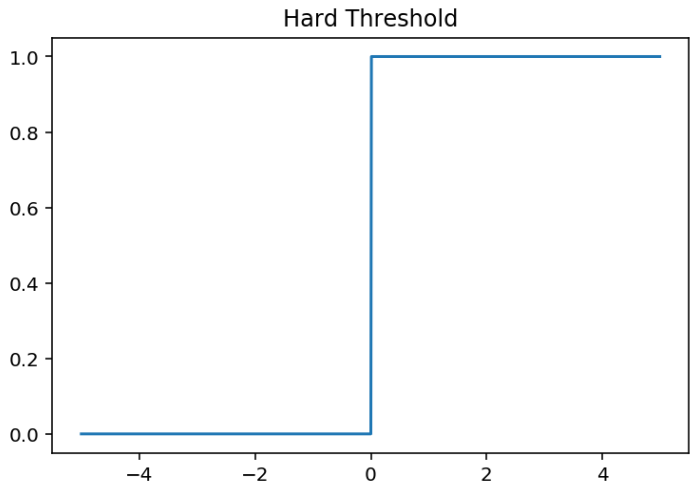


Types of Activation Functions

- Hard threshold

$$g(s) = 1 \text{ iff } s > 0, \text{else } 0$$

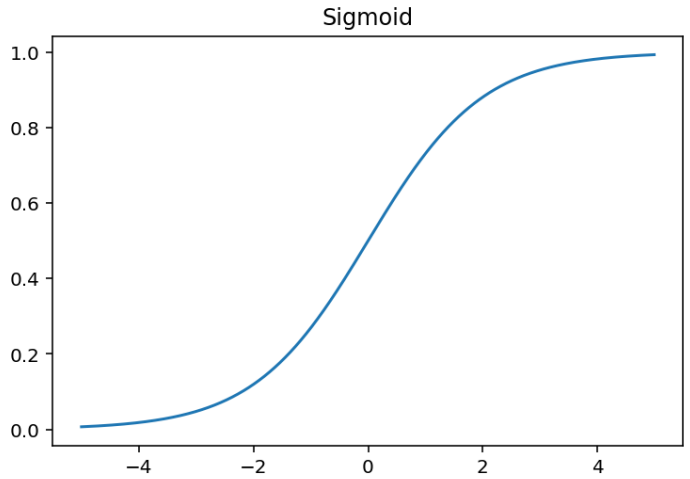
Non-differentiable!



- Sigmoid threshold

$$g(s) = \frac{1}{1 + e^{-s}}$$

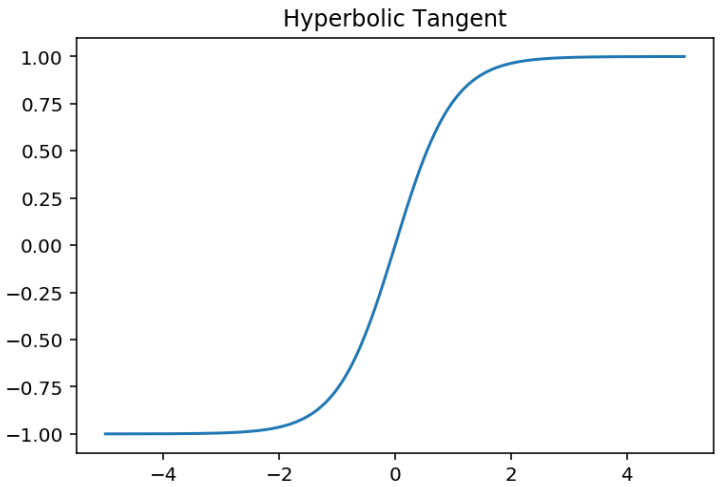
$$g'(s) = g(s)(1 - g(s))$$



Types of Activation Functions

- Hyperbolic Tangent (tanh)

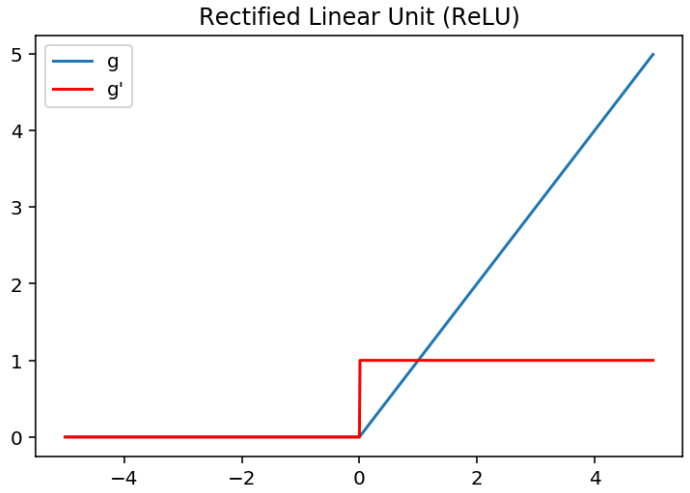
$$g(s) = \frac{e^{s} - e^{-s}}{e^{s} + e^{-s}}$$



- Rectified Linear Unit (ReLU)

$$g(s) = \max(0, s)$$

$$g'(s) = \begin{cases} 1, & s > 0 \\ 0, & otherwise \end{cases}$$



This is just the hard threshold.
Leads to sparse activations!

When poll is active respond at **PollEv.com/492**

Example

- Suppose a unit has:
  - $\mathbf{w} = [0.2, 0.3, 0.9]$
  - $b = 0.5$
  - Sigmoid activation function

$$\sigma(s) = \frac{1}{1 + e^{-s}}$$

- What is the output y for input vector $\mathbf{x} = [0.5, 0.6, 0.1]$?

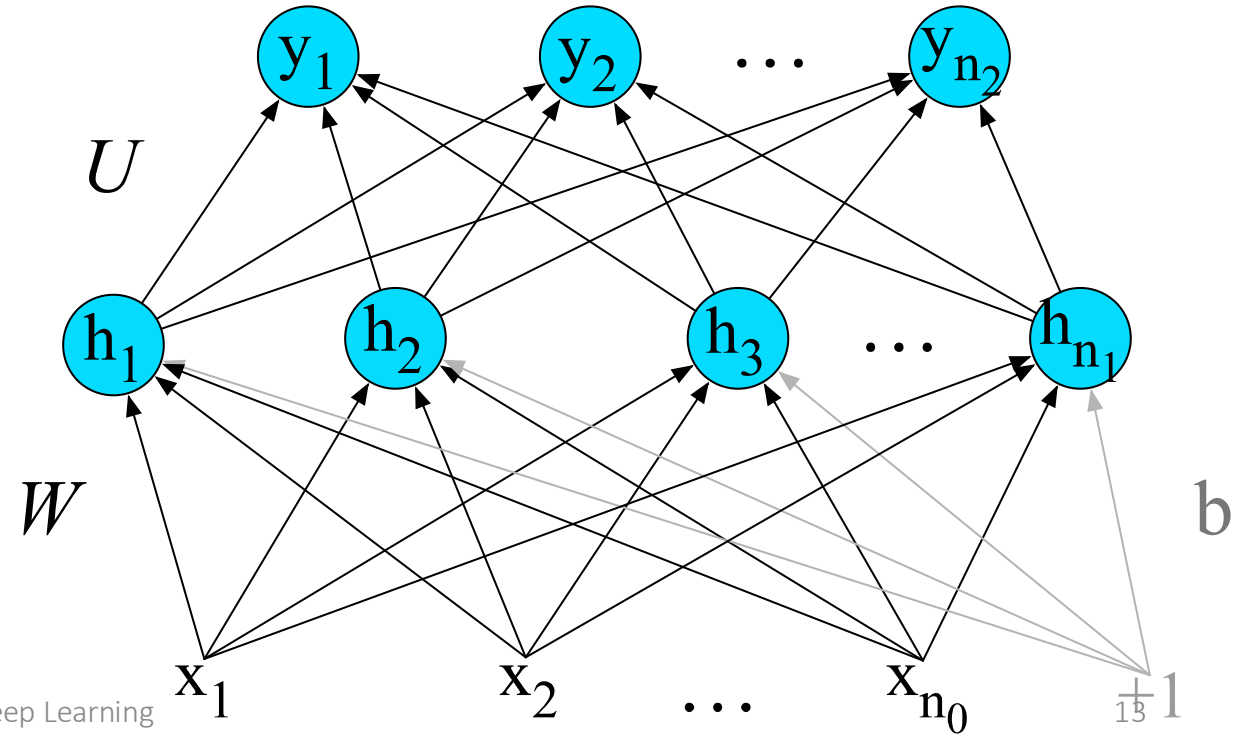
- x_i: ith input
- W, U: weight matrices

Feedforward Neural Networks (FNN)

- Multilayer network in which the units are connected with no cycles
  - Outputs from units in each layer are passed to units in the next higher layer
  - No outputs are passed back to lower layers

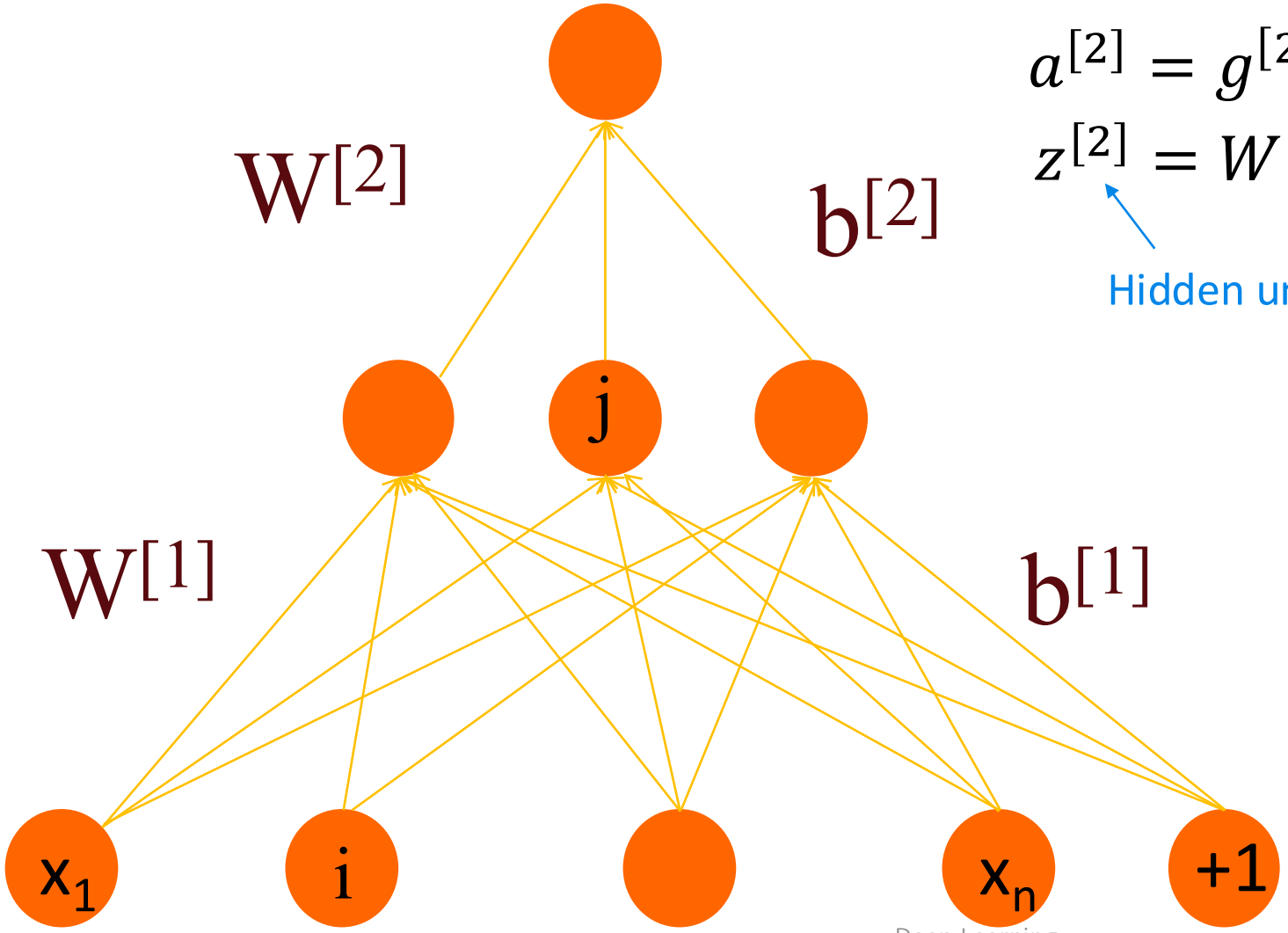
- Each layer is fully-connected
  - Link between every pair of units from 2 adjacent layers

- Also known as:
  - Multi-layer perceptron (**MLP**)
  - Fully-connected Neural Net (**FCNN**)



- x_i: ith input
- W, U: weight matrices

Multi-layer Notation



$$y = a^{[2]}$$

$$a^{[2]} = g^{[2]}(z^{[2]})$$

$$z^{[2]} = W^{[2]}a^{[1]} + b^{[2]}$$

Hidden unit, often denoted with letter z or h

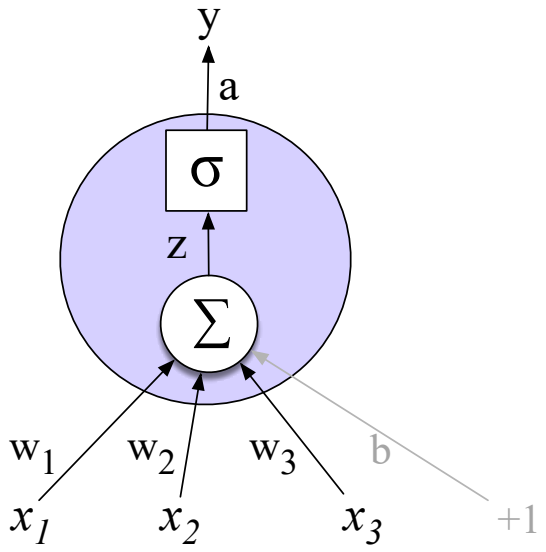
$$a^{[1]} = g^{[1]}(z^{[1]})$$

$$z^{[1]} = W^{[1]}a^{[0]} + b^{[1]}$$

$a^{[0]}$

Multi Layer Notation

$$
\begin{aligned}
z^{[1]} &= W^{[1]}a^{[0]} + b^{[1]} \\
a^{[1]} &= g^{[1]}(z^{[1]}) \\
z^{[2]} &= W^{[2]}a^{[1]} + b^{[2]} \\
a^{[2]} &= g^{[2]}(z^{[2]}) \\
\hat{y} &= a^{[2]}
\end{aligned}
$$



for i **in** 1..n

$$
\begin{aligned}
z^{[i]} &= W^{[i]}\, a^{[i-1]} + b^{[i]} \\
a^{[i]} &= g^{[i]}(z^{[i]}) \\
\hat{y} &= a^{[n]}
\end{aligned}
$$

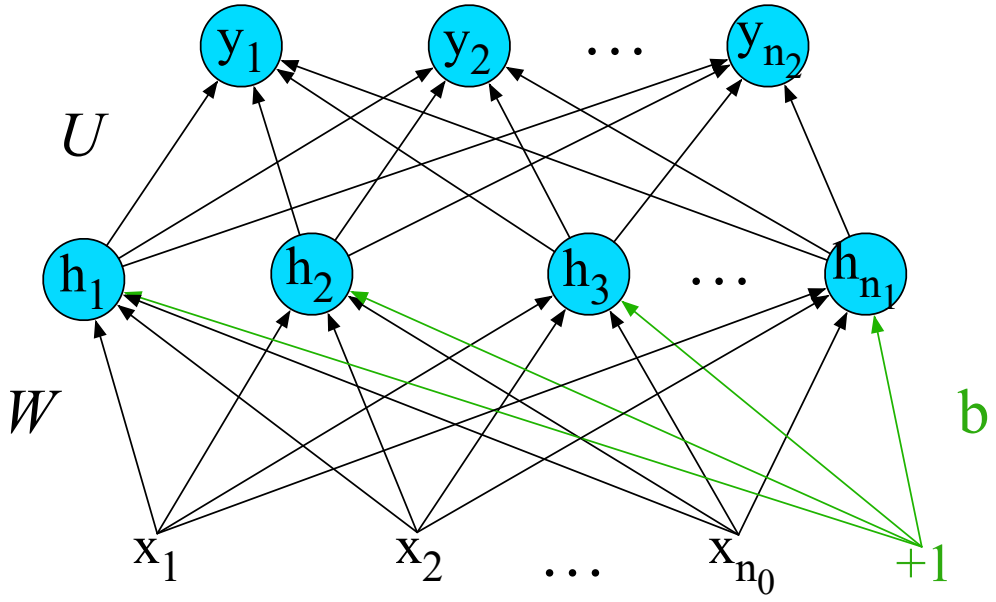
Replacing the bias unit

- Let's switch to a notation without the bias unit
- Just a notational change

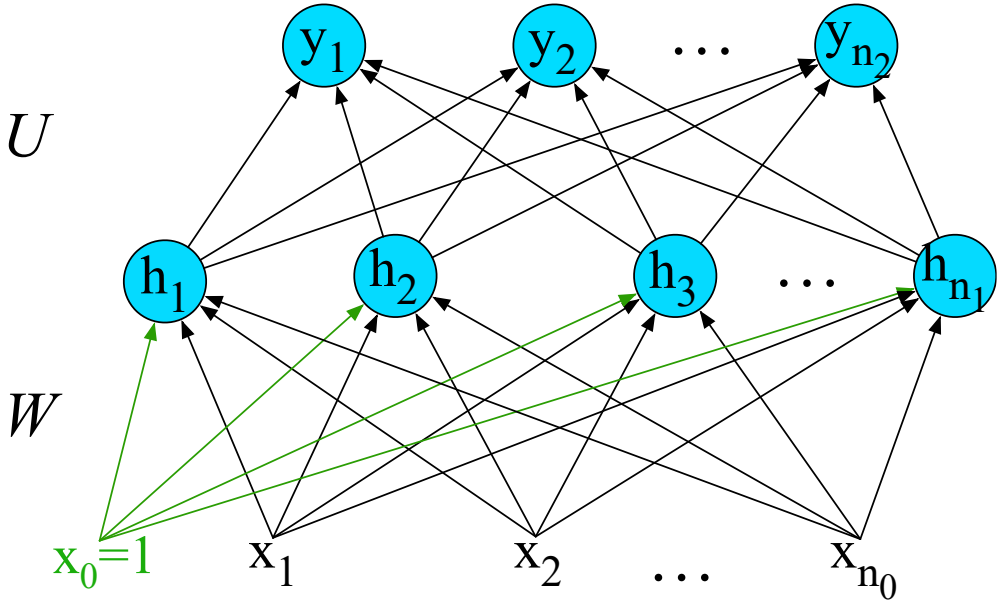
1. Add a dummy node $a_0=1$ to each layer
2. Its weight w_0 will be the bias
3. So input layer $a^{[0]}_0=1$,
   - And $a^{[1]}_0=1$, $a^{[2]}_0=1$,…

Replacing the bias unit

Instead of:



We'll do this:



Replacing the bias unit

- Instead of:

$$x = x_1, x_2, \ldots, x_{n0}$$

$$h = g\left(\sum_{i=1}^{n_0} W_{ji} x_i + b_j\right)$$

$$h = g(Wx + b)$$

We'll do this:

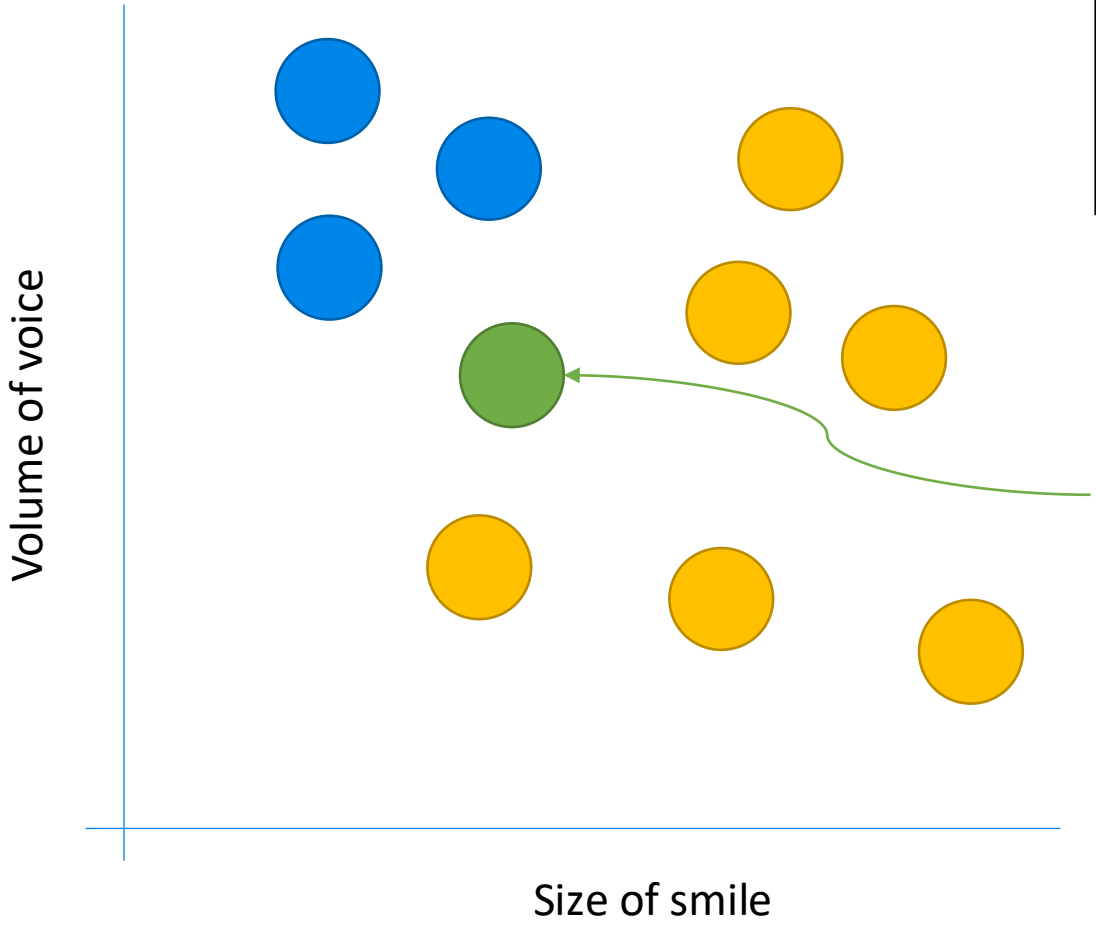
$$x = \boxed{x_0}, x_1, x_2, \ldots, x_{n0}$$

$$h = g\left(\sum_{i=0}^{n_0} W_{ji} x_i + b_j\right)$$

$$h = g(Wx)$$

Easier to handle!
(do this only for the input layer)

A toy example: Am I happy?



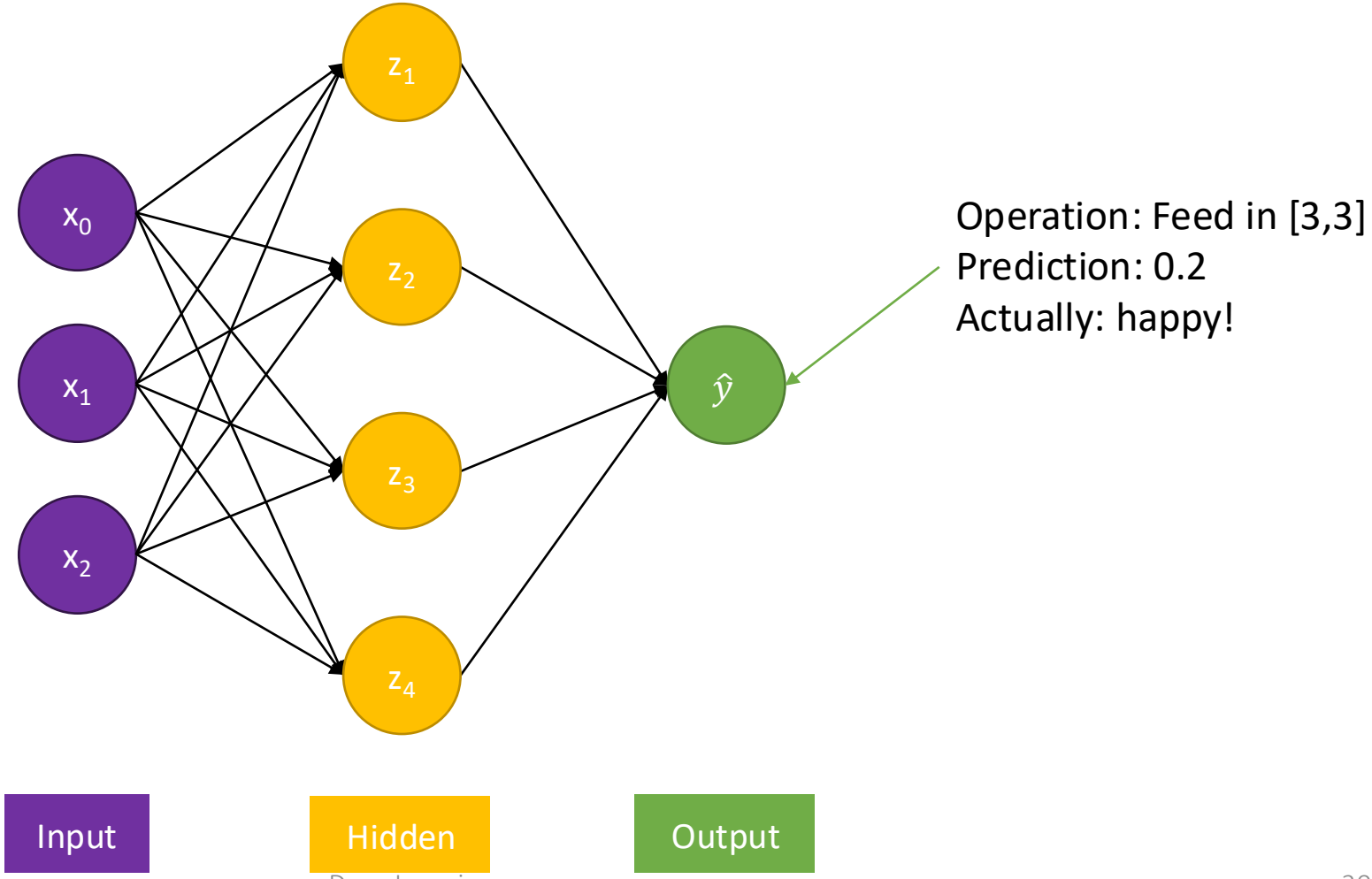
Legend:

- Happy
- Not happy

Variable change:
Let **Happy** be 1 and **Not happy** be 0

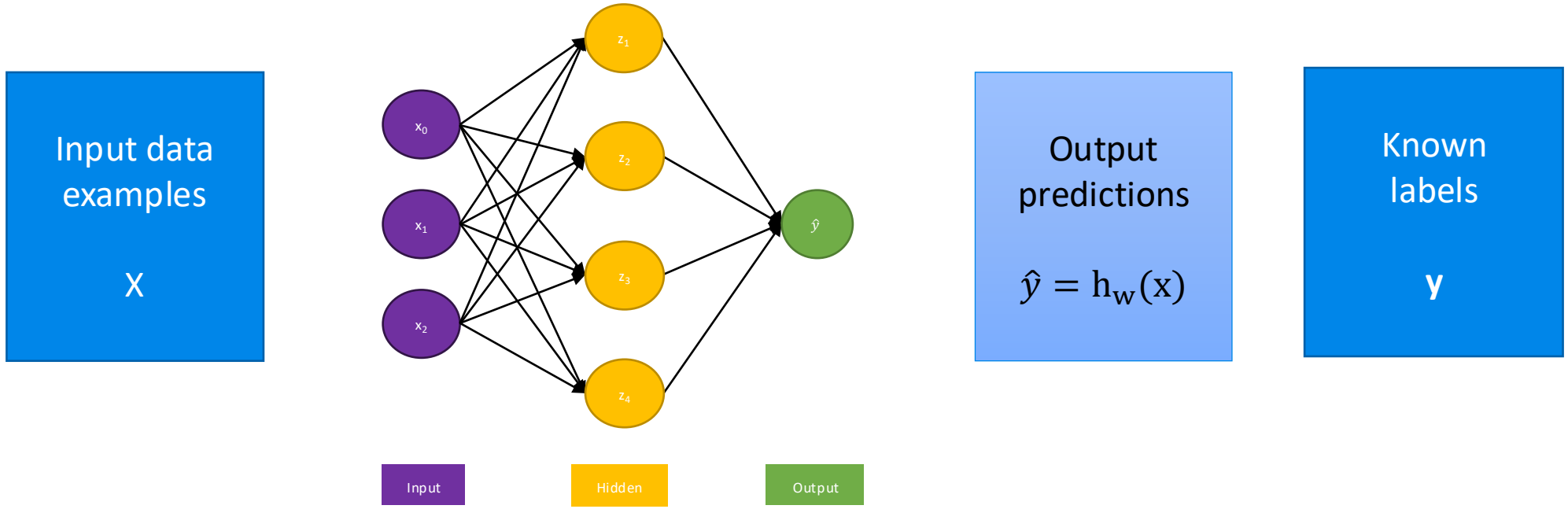
A new example
Location: [3, 3]

A toy example: Am I happy?



How close was I?

- Common approach: binary cross entropy loss
  - Used when known labels are binary (only 2 classes)



Cross Entropy

- Remember entropy?
  - Entropy = measure of uncertainty (higher entropy, more uncertainty)
  - $H(V) = -\sum_k P(v_k) log_2 P(v_k)$

- Related idea: cross entropy
  - It measures how well one distribution B approximates a true distribution A

$$H(A, B) = -\sum_k P(a_k)\, log_2 P(b_k)$$

k: possible outcomes (classes) of the random variable

  - In supervised learning:
    - A represents the **true** (data-generating) distribution
    - B is the model's **predicted** distribution

Binary Cross Entropy Loss

Notation:

- $\boldsymbol{X}$: the input matrix
- $\boldsymbol{z_i}$: the hidden units
- $\boldsymbol{s_i}$: the output layer, pre-nonlinearity
- $\boldsymbol{W}$: the weight matrix
- $\hat{\boldsymbol{y}}/\mathrm{h_w(X)}$: the estimated labels
- $\boldsymbol{y}$: the "true" labels

- We can use cross entropy as a loss
  - y – the true labels (labeled 0 or 1)
  - $h_W(x)$ - the labels predicted by the neural network

$$Loss(W) = -\frac{1}{n}\sum_{i=1}^{n} \underline{y^{(i)} \log\left(h_W\left(x^{(i)}\right)\right)} + \left(\underline{1 - y^{(i)}}\right)\log\left(\underline{1 - h_W\left(x^{(i)}\right)}\right)$$

Cross entropy for the true labels with class 1

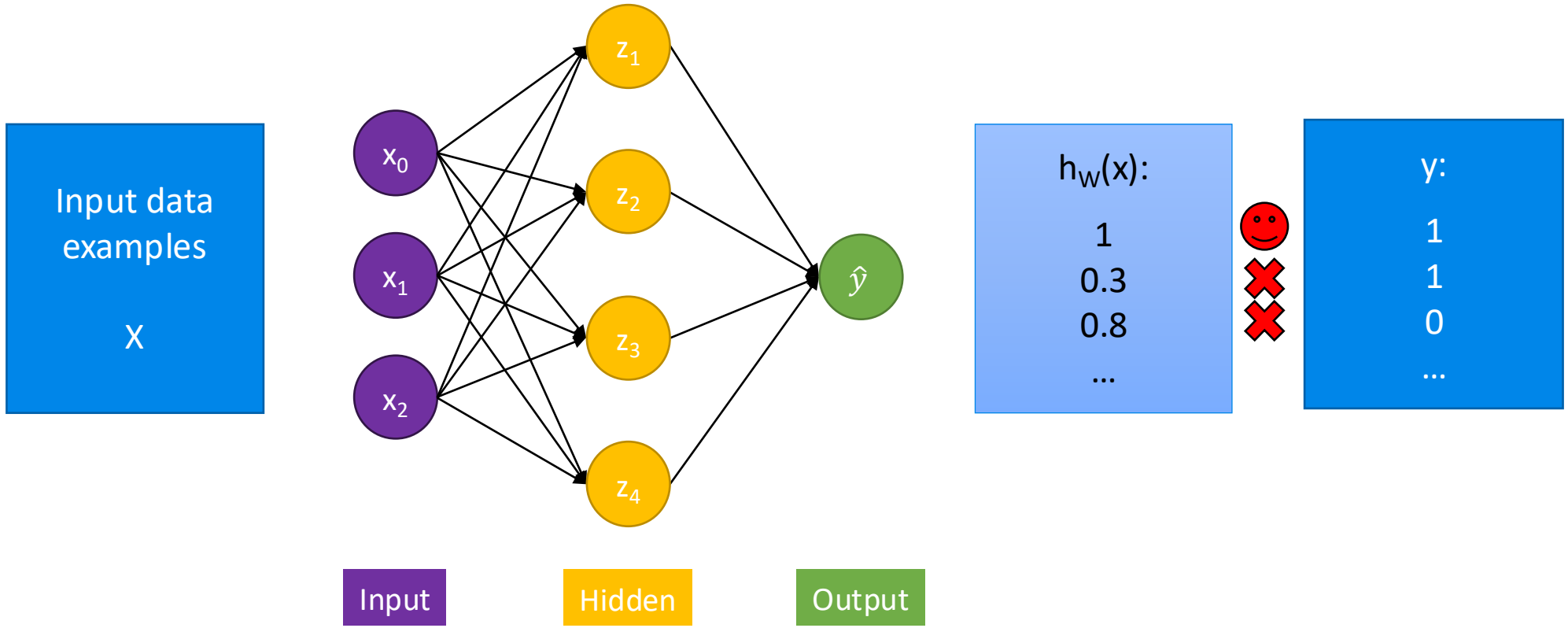
Cross entropy for the true labels with class 0

How close was I?

- Common approach: binary cross entropy loss

Notation:

- $\boldsymbol{X}$: the input matrix
- $\boldsymbol{z_i}$: the hidden units
- $\boldsymbol{s_i}$: the output layer, pre-nonlinearity
- $\boldsymbol{W}$: the weight matrix
- $\hat{\boldsymbol{y}}/\mathrm{h_w(X)}$: the estimated labels
- $\boldsymbol{y}$: the "true" labels

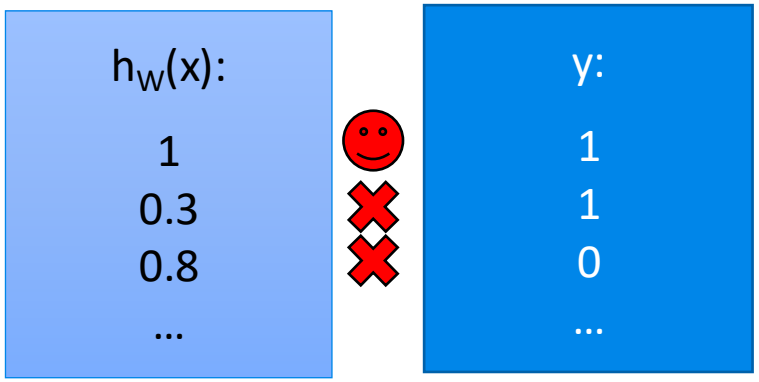


How close was I?

- Common approach: binary cross entropy loss

Notation:

- $\boldsymbol{X}$: the input matrix
- $\boldsymbol{z_i}$: the hidden units
- $\boldsymbol{s_i}$: the output layer, pre-nonlinearity
- $\boldsymbol{W}$: the weight matrix
- $\hat{\boldsymbol{y}}/\mathrm{h_w(X)}$: the estimated labels
- $\boldsymbol{y}$: the "true" labels



Formulate loss:

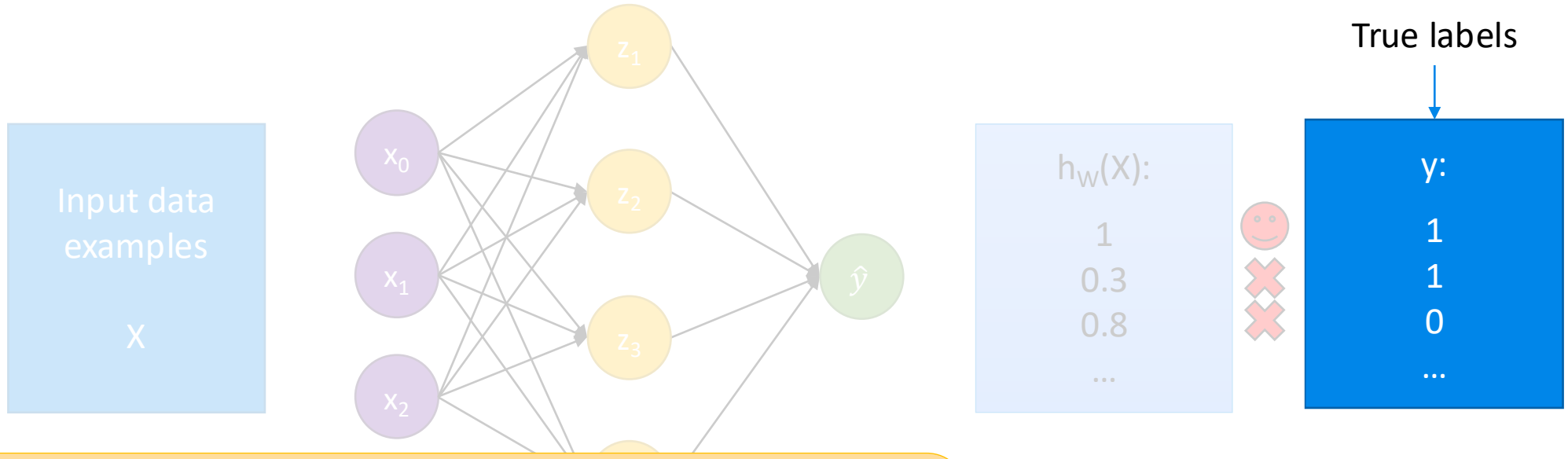
$$Loss(W) = -\frac{1}{n}\sum_i y^{(i)} \log\left(h_W\left(x^{(i)}\right)\right) + \left(1 - y^{(i)}\right)\log\left(1 - h_W\left(x^{(i)}\right)\right)$$

How close was I?

- Common approach: binary cross entropy loss

Notation:

- $\boldsymbol{X}$: the input matrix
- $\boldsymbol{z_i}$: the hidden units
- $\boldsymbol{s_i}$: the output layer, pre-nonlinearity
- $\boldsymbol{W}$: the weight matrix
- $\hat{\boldsymbol{y}}/\mathrm{h_w(X)}$: the estimated labels
- $\boldsymbol{y}$: the "true" labels



Formulate loss:

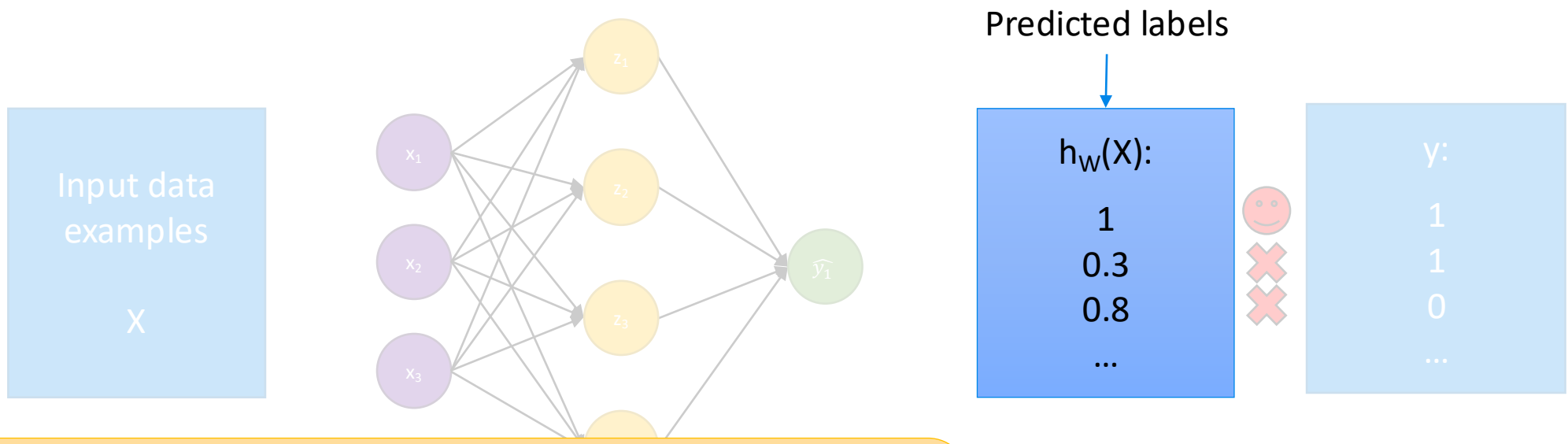
$$Loss(W) = -\frac{1}{n}\sum_i \boldsymbol{y^{(i)}} \log\left(h_W\left(x^{(i)}\right)\right) + \left(1 - \boldsymbol{y^{(i)}}\right)\log\left(1 - h_W\left(x^{(i)}\right)\right)$$

How close was I?

- Common approach: binary cross entropy loss

Notation:

- $\boldsymbol{X}$: the input matrix
- $\boldsymbol{z_i}$: the hidden units
- $\boldsymbol{s_i}$: the output layer, pre-nonlinearity
- $\boldsymbol{W}$: the weight matrix
- $\hat{\boldsymbol{y}}/\mathrm{h_w(X)}$: the estimated labels
- $\boldsymbol{y}$: the "true" labels



Formulate loss:

$$Loss(W) = -\frac{1}{n}\sum_i y^{(i)} \log\left(\boldsymbol{h_W}\left(\boldsymbol{x}^{(i)}\right)\right) + \left(1 - y^{(i)}\right)\log\left(1 - \boldsymbol{h_W}\left(\boldsymbol{x}^{(i)}\right)\right)$$

How close was I?

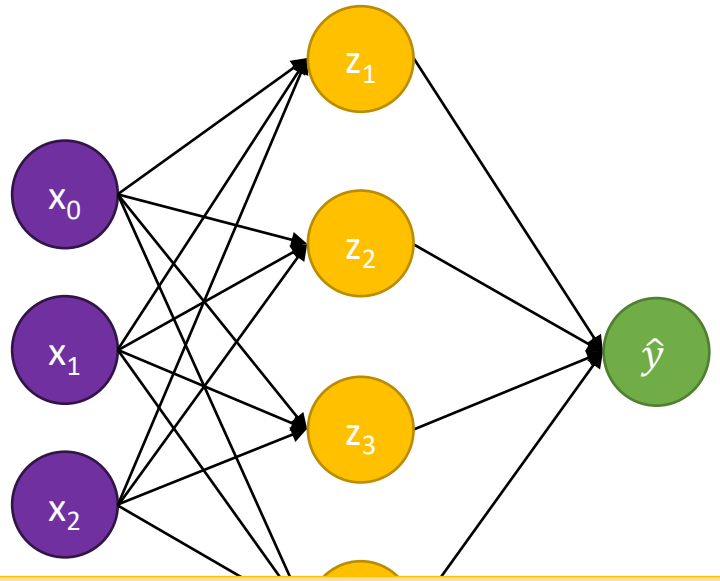
- Common approach: binary cross entropy loss

Notation:

- $\boldsymbol{X}$: the input matrix
- $\boldsymbol{z_i}$: the hidden units
- $\boldsymbol{s_i}$: the output layer, pre-nonlinearity
- $\boldsymbol{W}$: the weight matrix
- $\hat{\boldsymbol{y}}/\mathrm{h_w(X)}$: the estimated labels
- $\boldsymbol{y}$: the "true" labels

Input data examples

X



Items that are actually happy

$h_W(X)$:		y:
1	☺	1
0.3	✗	1
0.8	✗	0
…		…

Formulate loss:

$$Loss(W) = -\frac{1}{n}\sum_i \boldsymbol{y^{(i)}\, log\left(h_W\left(x^{(i)}\right)\right)} + \left(1 - y^{(i)}\right) log\left(1 - h_W\left(x^{(i)}\right)\right)$$

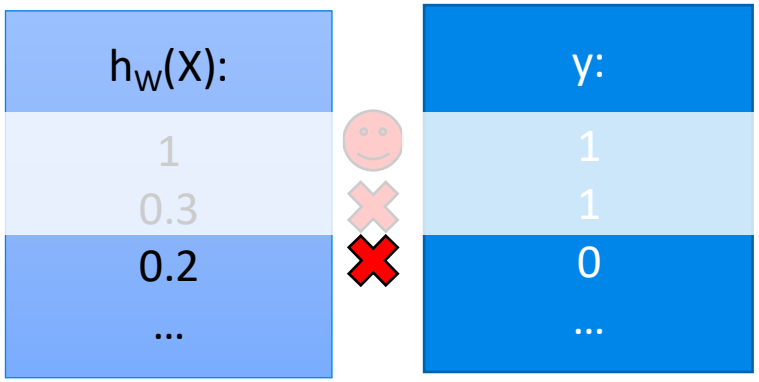
How close was I?

- Common approach: binary cross entropy loss

Notation:

- $\boldsymbol{X}$: the input matrix
- $\boldsymbol{z_i}$: the hidden units
- $\boldsymbol{s_i}$: the output layer, pre-nonlinearity
- $\boldsymbol{W}$: the weight matrix
- $\hat{\boldsymbol{y}}/\mathrm{h_w(X)}$: the estimated labels
- $\boldsymbol{y}$: the "true" labels

Items that are actually not happy



Formulate loss:

$$Loss(W) = -\frac{1}{n}\sum_i y^{(i)} \log\left(h_W(x^{(i)})\right) + \left(1 - y^{(i)}\right)\log\left(1 - h_W(x^{(i)})\right)$$

How close was I?

- Common approach: mean squared error (MSE) loss (how happy am I?)

Notation:

- $\boldsymbol{X}$: the input matrix
- $\boldsymbol{z_i}$: the hidden units
- $\boldsymbol{s_i}$: the output layer, pre-nonlinearity
- $\boldsymbol{W}$: the weight matrix
- $\hat{\boldsymbol{y}}/\mathrm{h_w(X)}$: the estimated labels
- $\boldsymbol{y}$: the "true" labels

Input data examples

X

$h_W(X)$:

94
32
25
…

y:

100
85
10
…

Formulate loss:

$$Loss(W) = \frac{1}{n}\sum_i \left(y^{(i)} - h_W\left(x^{(i)}\right)\right)^2$$

Now we can train the model!

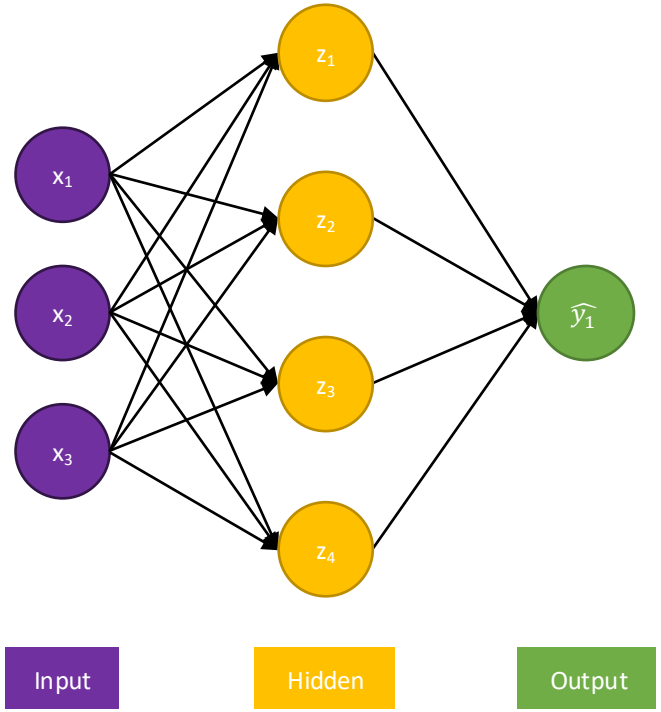
Notation:

- $\boldsymbol{X}$: the input matrix
- $\boldsymbol{z_i}$: the hidden units
- $\boldsymbol{s_i}$: the output layer, pre-nonlinearity
- $\boldsymbol{W}$: the weight matrix
- $\hat{\boldsymbol{y}}/\mathrm{h_w(x)}$: the estimated labels
- $\boldsymbol{y}$: the "true" labels

- We would like to *minimize* loss

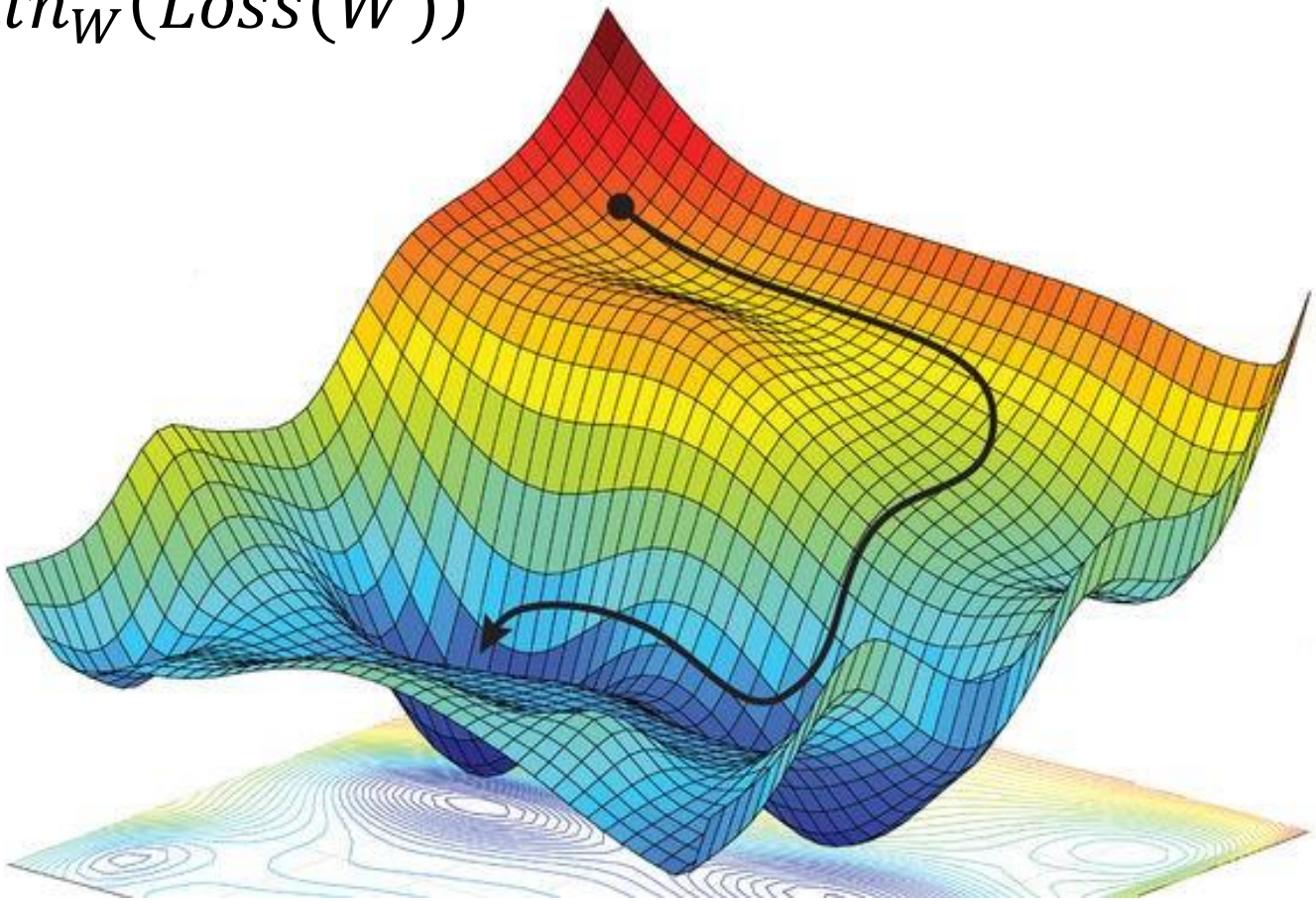
$$W^* = argmin_w \underbrace{\frac{1}{n}\sum_i \left(\mathcal{L}\left(h_W(x), y^{(i)}\right)\right)}_{Loss(W)}$$

$$W^* = argmin_W(Loss(W))$$



We need to find the right weights

$$W^* = argmin_W(Loss(W))$$



https://miro.medium.com/max/1200/1*mElyetzsTIJrNnKI8kTkCw.jpeg

Optimization Methods for Neural Networks

- Batch Gradient descent

- Stochastic gradient descent

Batch Gradient Descent

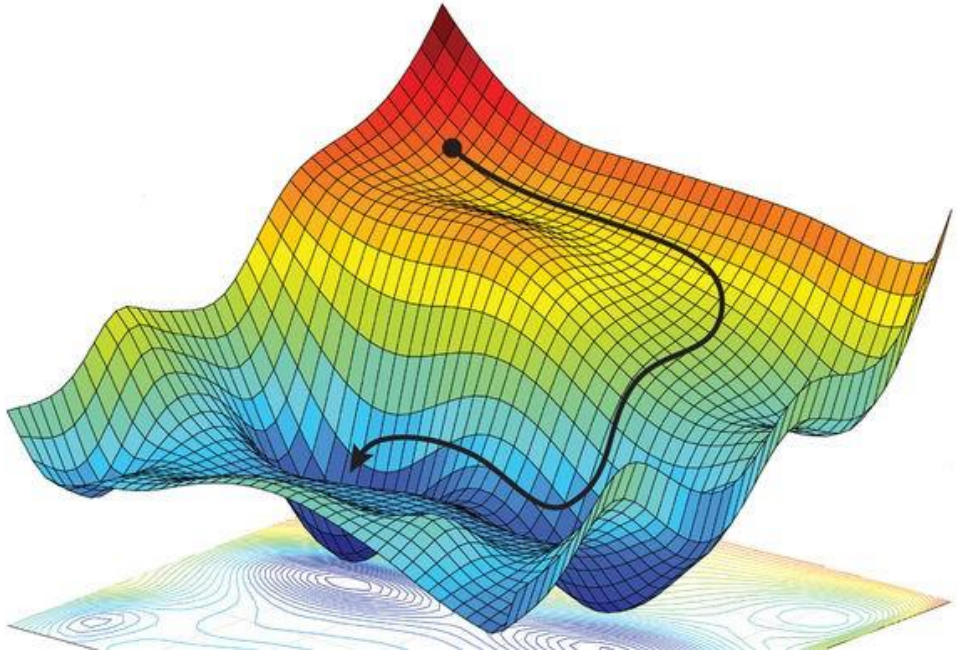
Notation:

- W: the weight matrix
- α: the learning rate

- Initialize weights randomly
- Loop until convergence:
  - Compute gradient: $\frac{\partial Loss(W)}{\partial W}$
  - Update weights: $W \leftarrow W - \alpha \frac{\partial Loss(W)}{\partial W}$
- Return weights

Learning rate

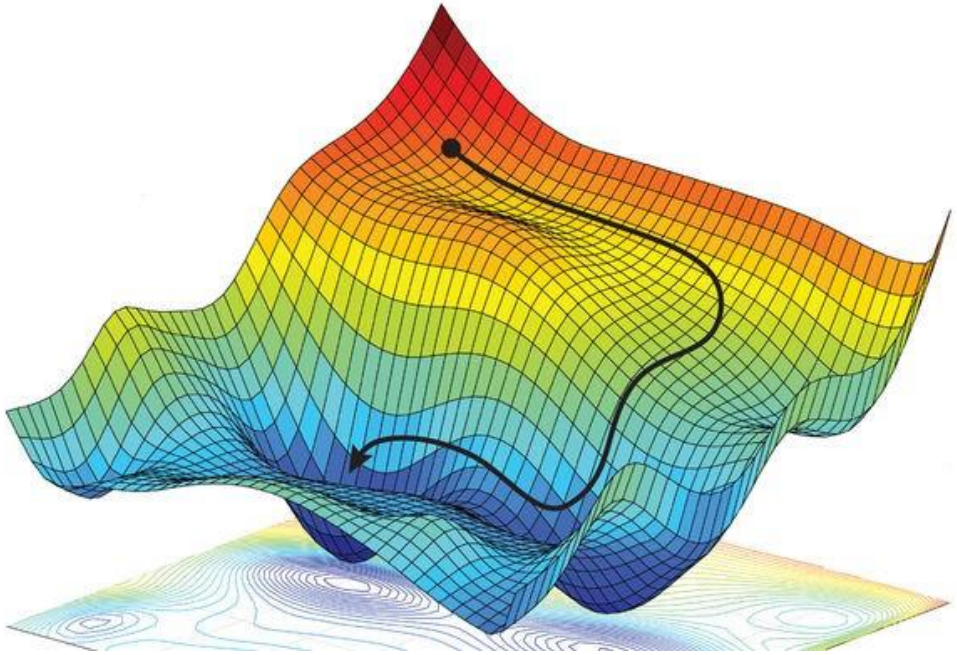
Move "downhill"



https://miro.medium.com/max/1200/1*mElyetzsTIJrNnKI8kTkCw.jpeg

When poll is active respond at **PollEv.com/492**

What happens if the learning rate is too small? Too big?



https://miro.medium.com/max/1200/1*mElyetzsTIJrNnKI8kTkCw.jpeg

Changing the learning rate

*Note: it is not necessary that the learning rate be fixed!

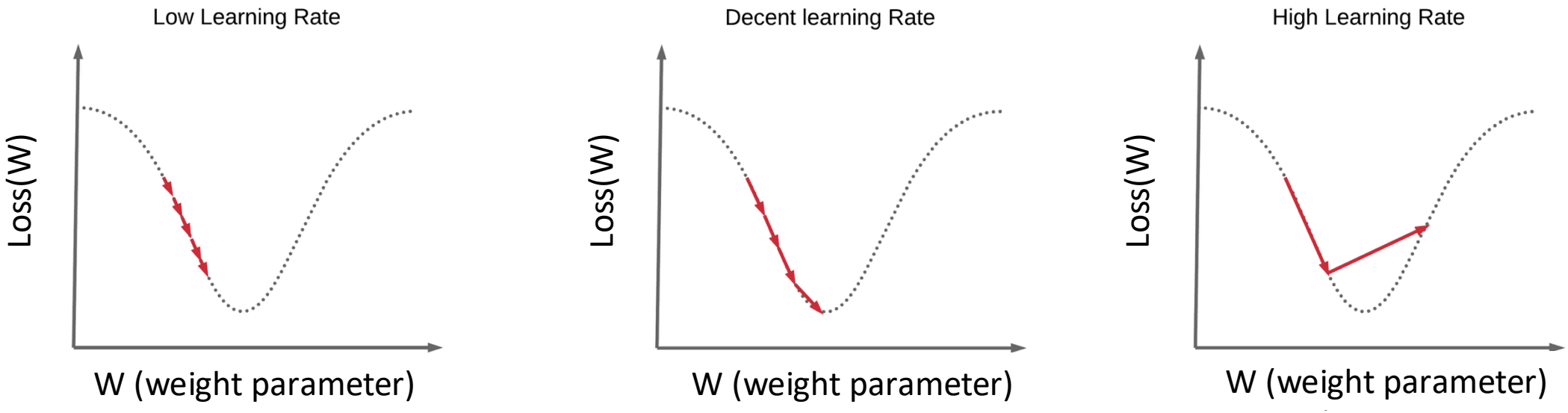


Image source: https://www.deeplearningwizard.com/deep_learning/boosting_models_pytorch/lr_scheduling/

Stochastic Gradient Descent (SGD)

- Batch gradient descent is expensive when N is large!
- Instead, SGD would update for every single example
- More efficient: approximate sum using a minibatch of examples
  - 32 / 64 / 128 common
  - Update weights with minibatch gradient

Minibatch Gradient Descent

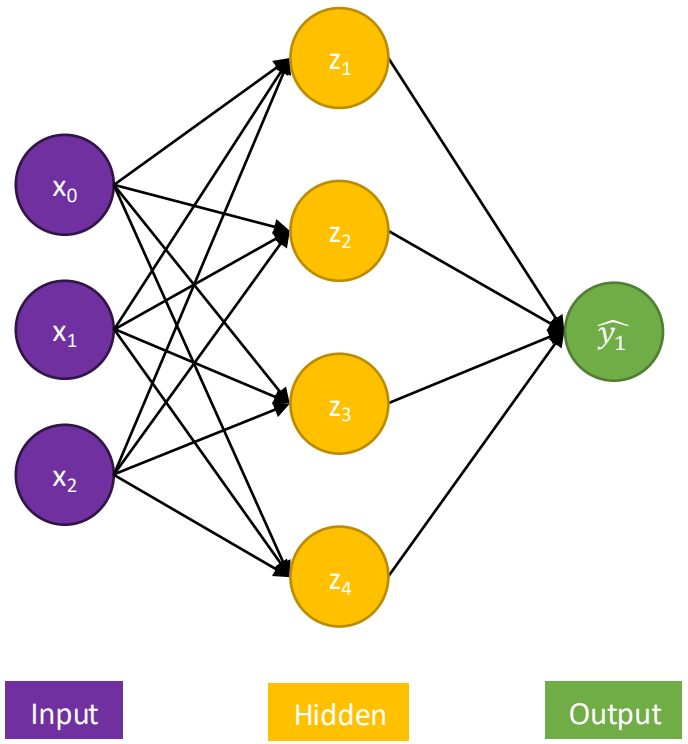
Initialize weights randomly
Loop until convergence:
    Shuffle dataset
    Loop over minibatch:
        Compute gradient for minibatch $\frac{\partial Loss\,(W)}{\partial W}$
        Update weights: $W \leftarrow W - \alpha \frac{\partial Loss(W)}{\partial W}$
Return weights

Backpropagation

Backpropagation

- Basic idea:
  - Compute effect of every weight on output
  - Work backwards from output to input

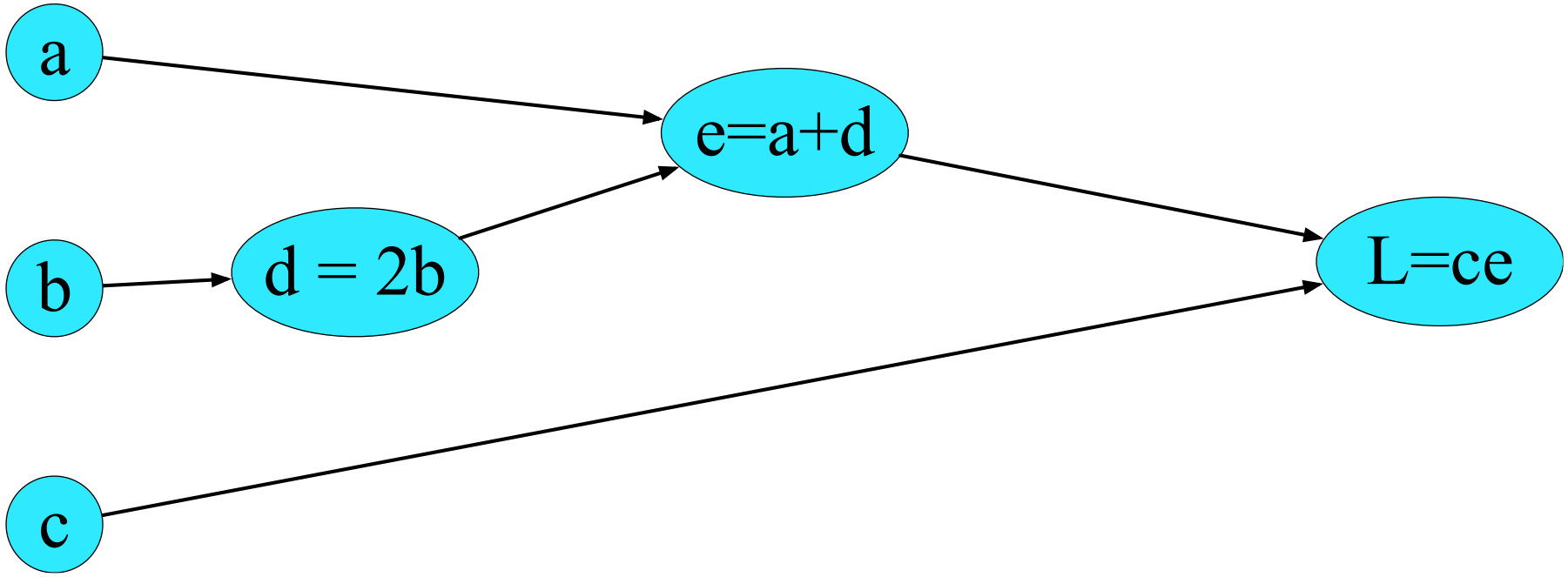
- If output is the wrong value – move weights in direction of gradient



Example of a computational graph

$$L(a,b,c) = c(a+2b)$$

$$\begin{aligned} d &= 2*b \\ e &= a+d \\ L &= c*e \end{aligned}$$



Backwards Differentiation

$$L(a,b,c) = c(a+2b)$$
$$d = 2*b$$
$$e = a+d$$
$$L = c*e$$

- Backward pass - we compute the derivatives that we'll need for the weight update

- It will help us obtain $\frac{\partial L}{\partial a}$, $\frac{\partial L}{\partial b}$, and $\frac{\partial L}{\partial c}$ to update the parameters via gradient descent

- Recall: the derivative $\frac{\partial L}{\partial a}$ tells us how much a small change in *a* affects *L*

Recall: The Chain Rule

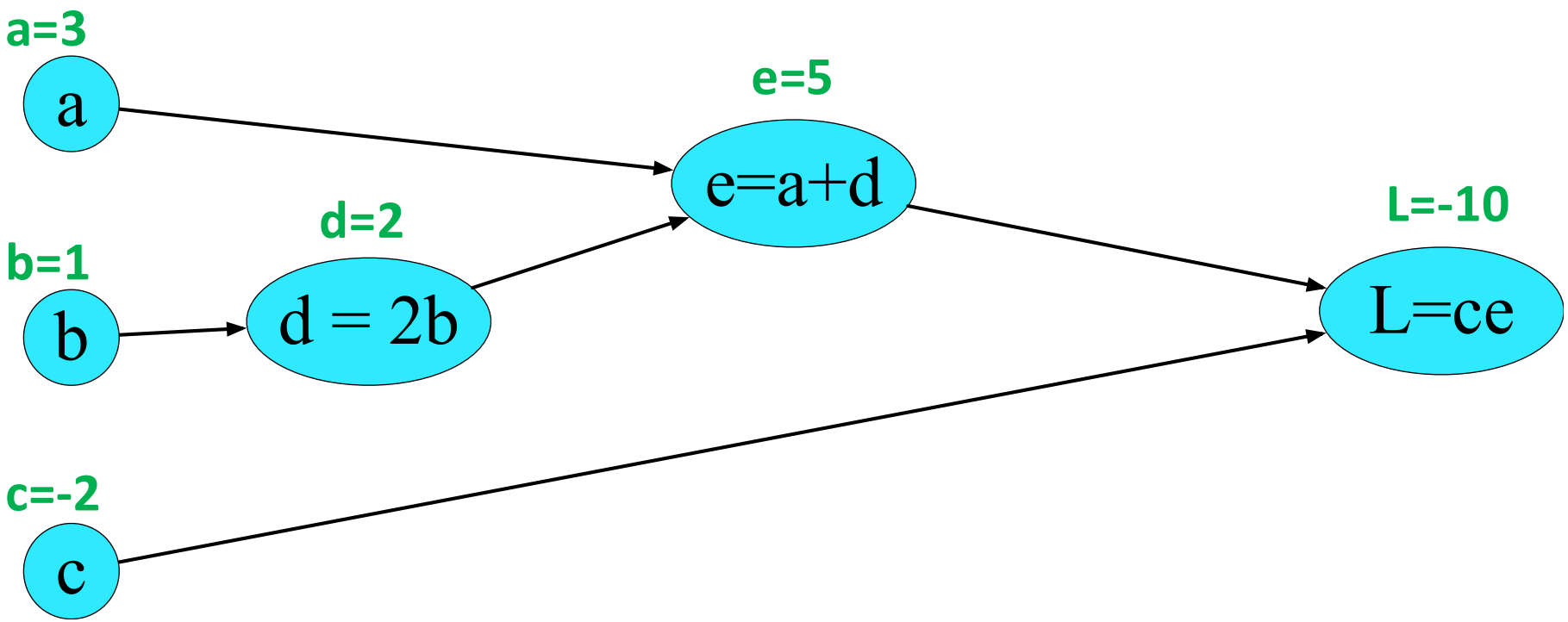
- Computing the derivative of a composite function:

- $f(x) = u(v(x))$ $$\frac{df}{dx} = \frac{du}{dv} \cdot \frac{dv}{dx}$$

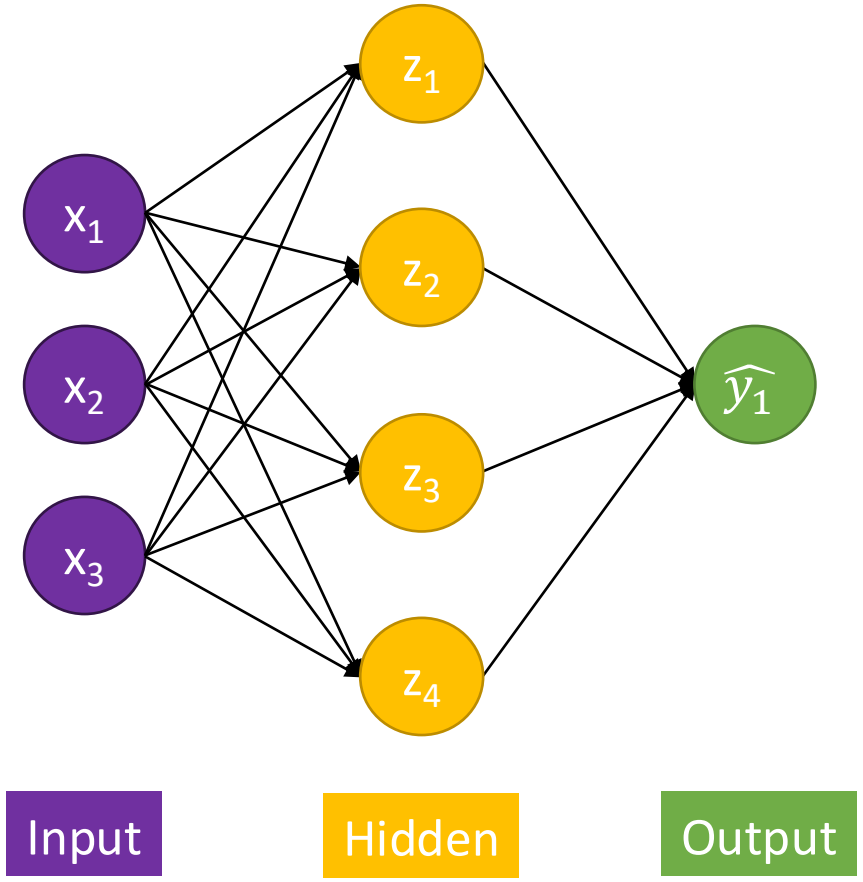
- $f(x) = u(v(w(x)))$ $$\frac{df}{dx} = \frac{du}{dv} \cdot \frac{dv}{dw} \cdot \frac{dw}{dx}$$

When poll is active respond at **PollEv.com/492**

Calculate $\frac{\partial L}{\partial a}$ using the computational graph

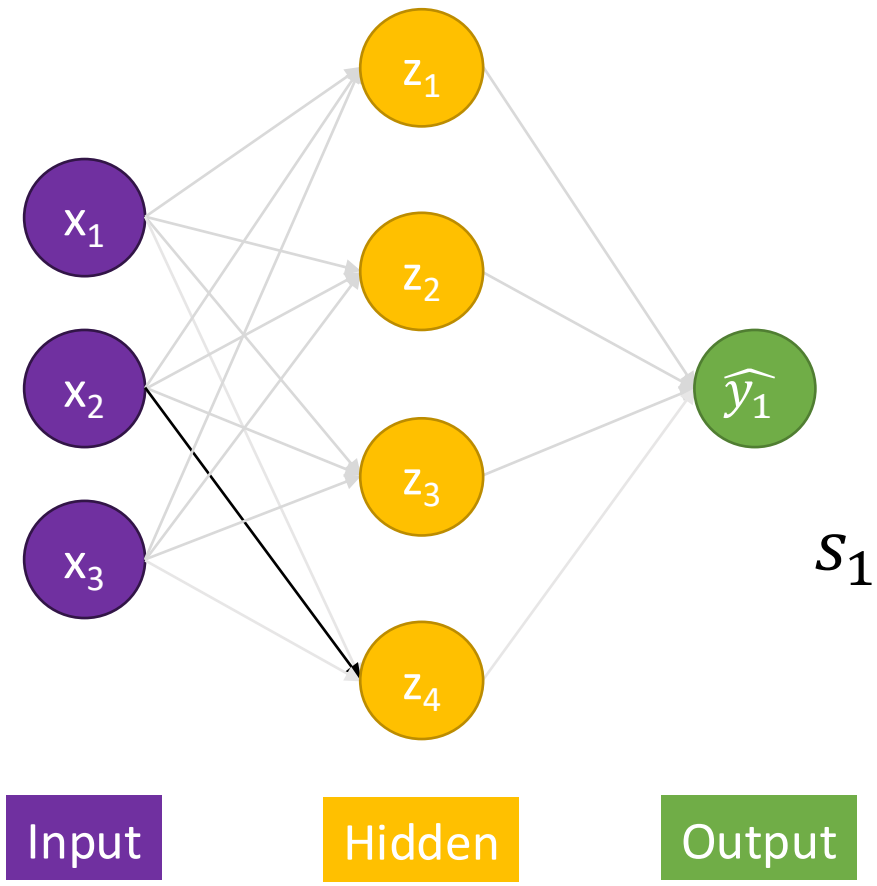


Back to training our 2-layer network



Let's step through it with focus on $w_{2,4}^{(1)}$

Back to training our 2-layer network

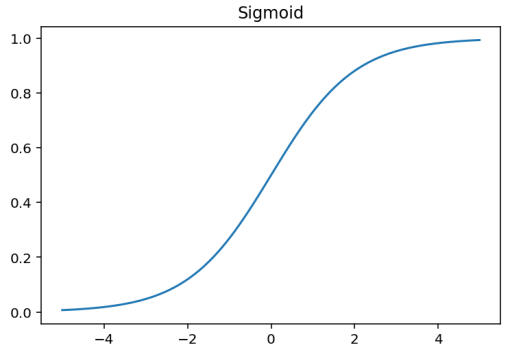


First, let's write down the forward pass equations containing $w_{2,4}^{(1)}$

$$z_4 = g\left(w_{0,4}^{(1)} + x_1 w_{1,4}^{(1)} + x_2 w_{2,4}^{(1)} + x_3 w_{3,4}^{(1)}\right)$$

$$s_1 = w_{0,1}^{(2)} + z_1 w_{1,1}^{(2)} + z_2 w_{2,1}^{(2)} + z_3 w_{3,1}^{(2)} + z_4 w_{4,1}^{(2)}$$

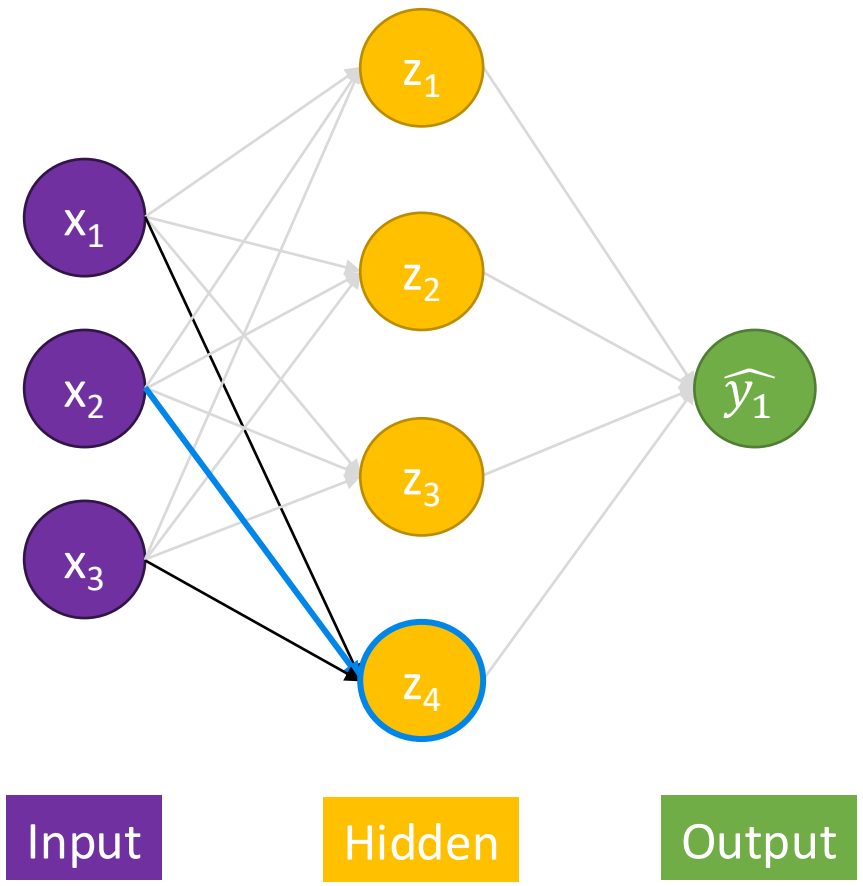
$$\hat{y}_1 = \sigma(s_1)$$



where $\sigma(s) = \frac{1}{1+e^{-s}}$

and g is ReLU

Back to training our 2-layer network

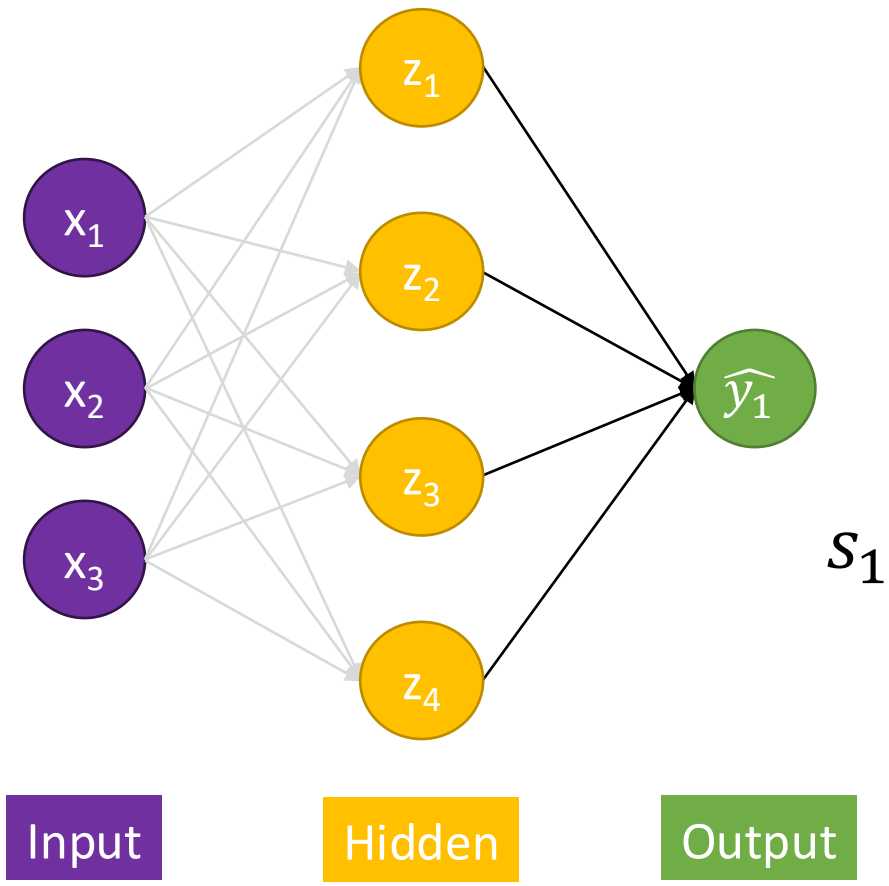


Let's step through it step by step with focus on $w_{2,4}^{(1)}$

$$z_4 = g\left(w_{0,4}^{(1)} + x_1 w_{1,4}^{(1)} + x_2 w_{2,4}^{(1)} + x_3 w_{3,4}^{(1)}\right)$$

where g is ReLU

Back to training our 2-layer network



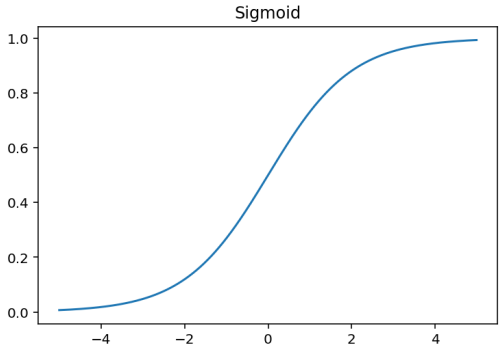
Let's step through it step by step with focus on z_4

Now with the output layer:

$$s_1 = w_{0,1}^{(2)} + z_1 w_{1,1}^{(2)} + z_2 w_{2,1}^{(2)} + z_3 w_{3,1}^{(2)} + z_4 w_{4,1}^{(2)}$$

$$\hat{y}_1 = \sigma(s_1)$$

where $\sigma(s) = \frac{1}{1+e^{-s}}$



Recall: Binary Cross Entropy Loss

- A measure for how far off the current answer is to the right answer

$$Loss(y, \hat{y}) = -[y \ \log \hat{y} + (1 - y) \log(1 - \hat{y})]$$

- Derivative:

$$\frac{\partial Loss}{\partial \hat{y}} = -\frac{y}{\hat{y}} + \frac{(1 - y)}{(1 - \hat{y})}$$

Updating $w_{2,4}^{(1)}$

$$Loss = -[y \log \hat{y}_1 + (1-y)\log(1-\hat{y}_1)]$$
$$\hat{y}_1 = \sigma(s_1)$$
$$s_1 = w_{0,1}^{(2)} + z_1 w_{1,1}^{(2)} + z_2 w_{2,1}^{(2)} + z_3 w_{3,1}^{(2)} + z_4 w_{4,1}^{(2)}$$
$$z_4 = g\left(w_{0,4}^{(1)} + x_1 w_{1,4}^{(1)} + x_2 w_{2,4}^{(1)} + x_3 w_{3,4}^{(1)}\right)$$

- We will need $\frac{\partial Loss}{\partial w_{2,4}^{(1)}}$
- Chain rule:

$$\frac{\partial Loss}{\partial w_{2,4}^{(1)}} = \frac{\partial Loss}{\partial \hat{y}_1} \cdot \frac{\partial \hat{y}_1}{\partial s_1} \cdot \frac{\partial s_1}{\partial z_4} \cdot \frac{\partial z_4}{\partial w_{2,4}^{(1)}}$$

Derivative of the Loss

Derivative of the activation (sigmoid)

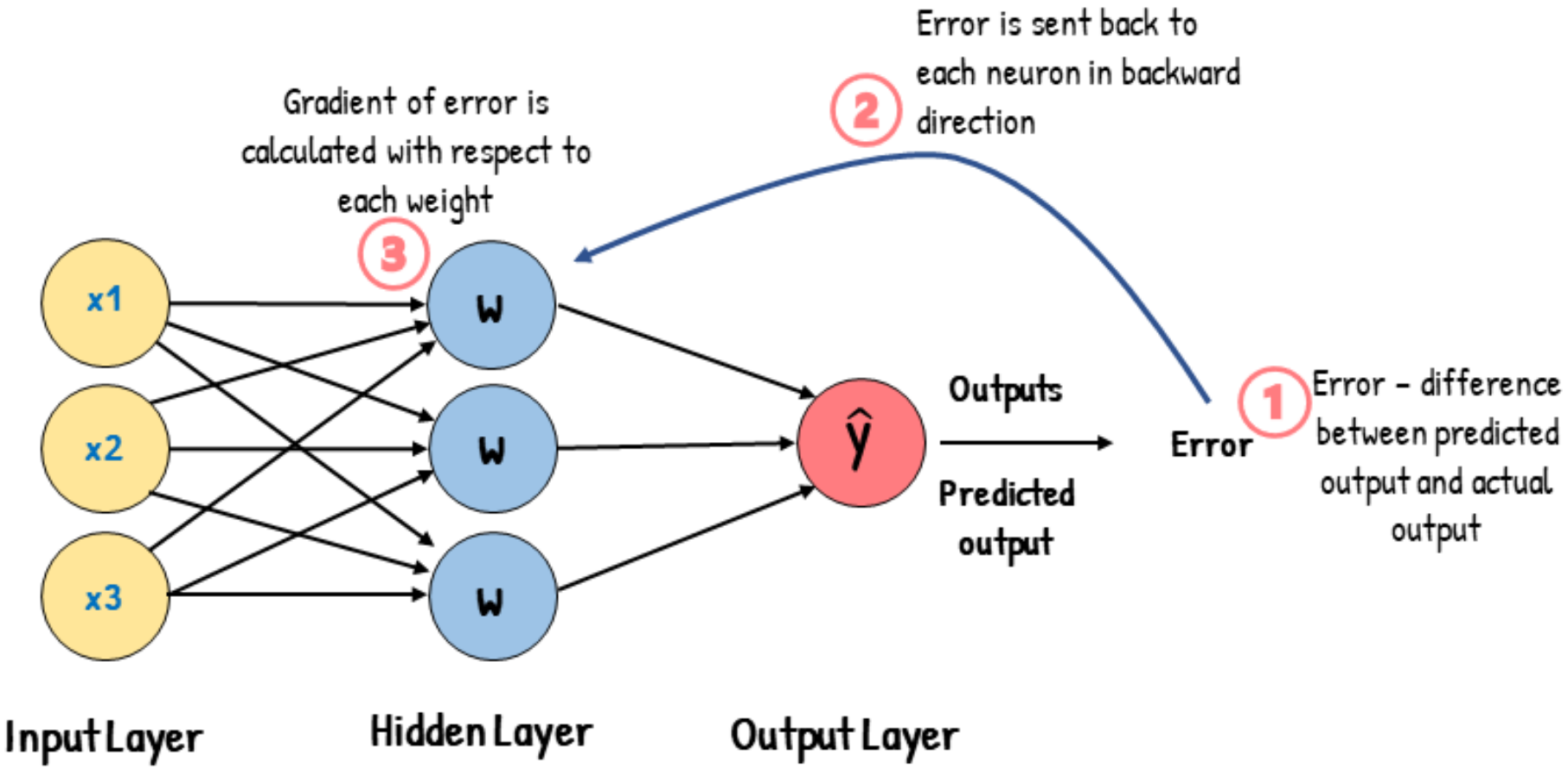
Derivative of the weighted sum

Derivative of activation (ReLU) and weighted sum

- Then, update rule: $w_{2,4}^{(1)} \leftarrow w_{2,4}^{(1)} - \alpha \frac{\partial Loss}{\partial w_{2,4}^{(1)}}$

We will do the same for every weight in the neural network

Summary: Backprop Computes Gradients needed for Gradient Descent Algorithm



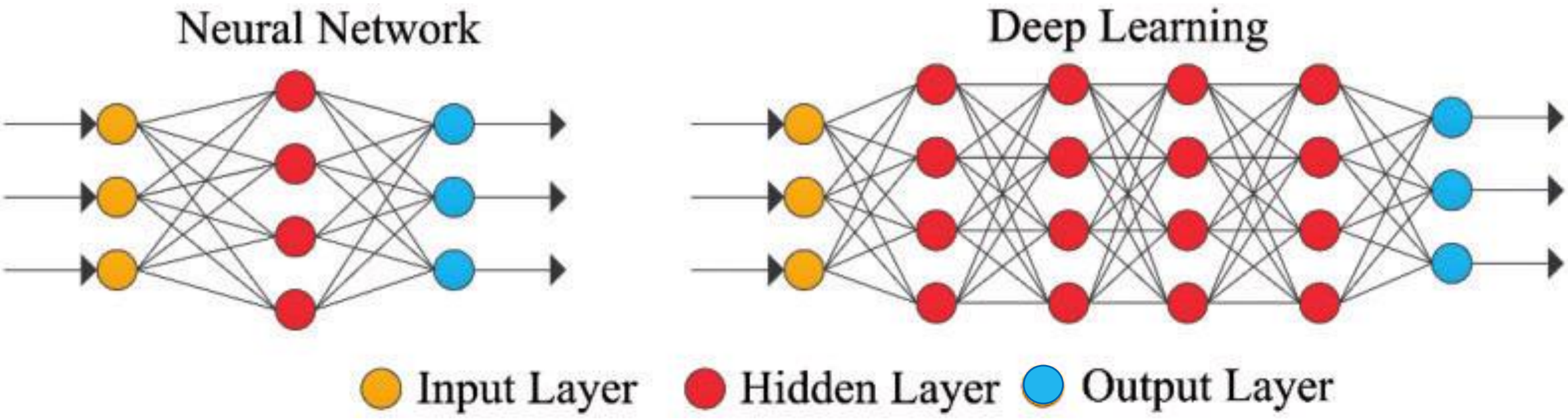
Source: https://www.analyticsvidhya.com/blog/2023/01/gradient-descent-vs-backpropagation-whats-the-difference/

Notes on Gradient Descent

- No convergence guarantees due to many local minima
- Problem of finding consistent weights is NP-complete
- Performance dependent on network structure
  - Need sufficient hidden nodes to express target
  - But too many nodes leads to overfitting + slow training!

Deep Learning: A Little Bit of History

From Neural Networks to Deep Learning

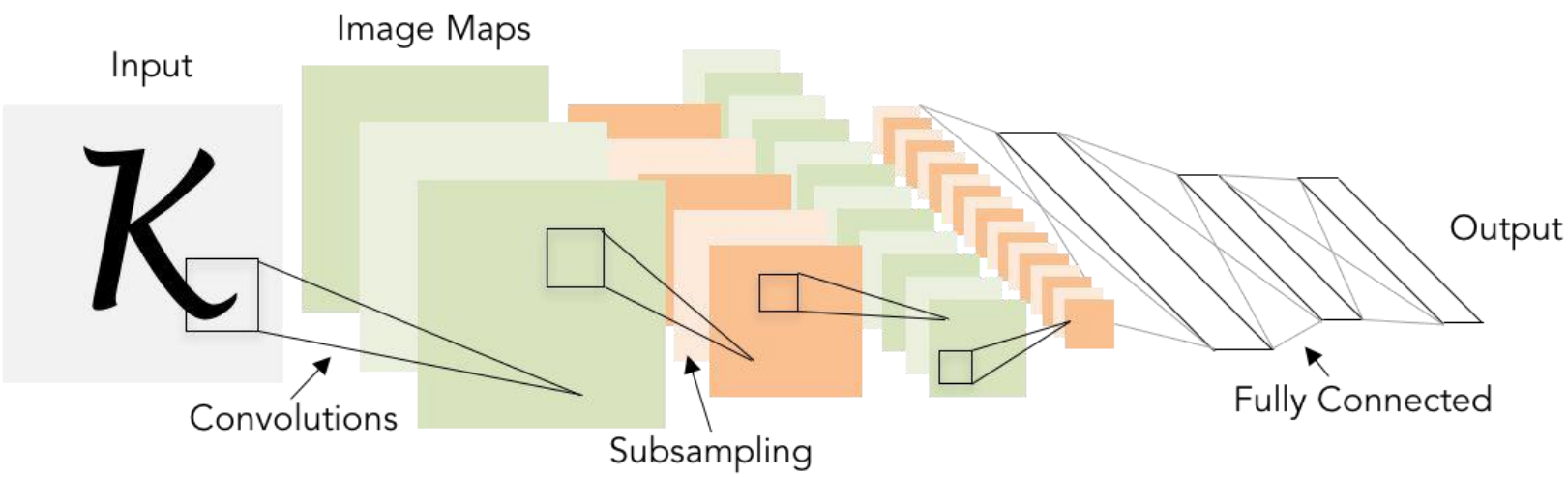


Source: Wanli Xing

A bit of history:

Gradient-based learning applied to document recognition

[LeCun, Bottou, Bengio, Haffner 1998]

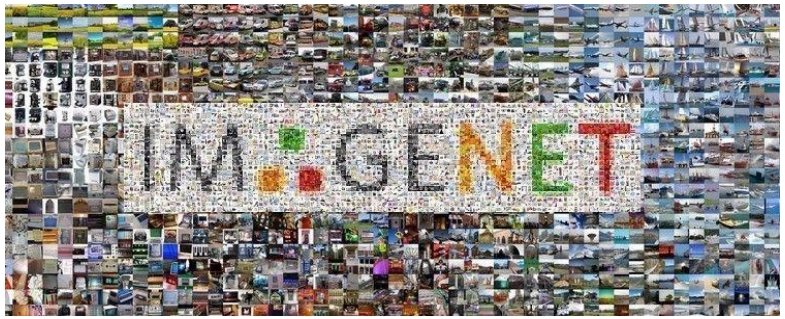


LeNet-5

LeNet-1 from 1993

A bit of history:

ImageNet Classification with Deep Convolutional Neural Networks *[Krizhevsky, Sutskever, Hinton, 2012]*



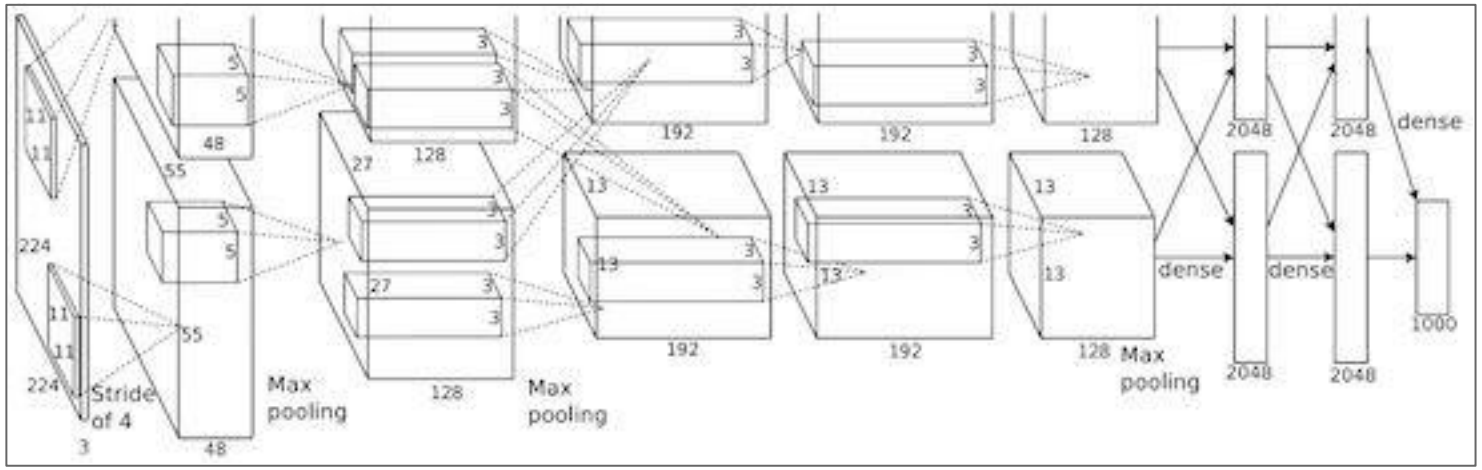


Figure copyright Alex Krizhevsky, Ilya Sutskever, and Geoffrey Hinton, 2012. Reproduced with permission.

"AlexNet"

AlexNet Results

Classification

Retrieval

Figures copyright Alex Krizhevsky, Ilya Sutskever, and Geoffrey Hinton, 2012. Reproduced with permission.

Convolutional Neural Nets (CNNs)

CNNs

original: Fukushima, Kunihiko. "Neocognitron: A self-organizing neural network model for a mechanism of pattern recognition unaffected by shift in position." *Biological cybernetics* 36.4 (1980): 193-202.

breakthrough: Yann Lecun, Léon Bottou, Yoshua Bengio, Patrick Haffner. Gradient-based learning applied to document recognition. Proceedings of the IEEE, 1998, 86 (11), pp.2278-2324. ff10.1109/5.726791ff. ffhal03926082f

Able to recognize spatial patterns in grid-like data.

Applications in computer vision

- Image classification
- Autonomous vehicles
- Facial recognition
- …

But is sometimes applied to other tasks as well (time series data: sequence of vectors)

- Sentence classification
- Speech recognition
- …

Image Processing: Problems w/ FCNN

RGB image: three integers (0-255) per pixel

Red Green Blue

- (0, 255, 0) is green
- (144, 83, 255) is lighter purple
- (60, 0, 168) is darker purple

1000x1000 RGB image:
1000x1000x3 = 3M integers

FCNN to input 1000x1000 RGB image, with 1000 hidden units
So $W^{[1]}$ would have shape (1000, 3M), so 3B weights!

- Great danger of overfitting
- Massive computational requirements
- The units in a FC layer don't know their spatial relationships.
- An image that's shifted a little bit appears completely different!

Design Principles of CNNs

A Dog

Spatial structure

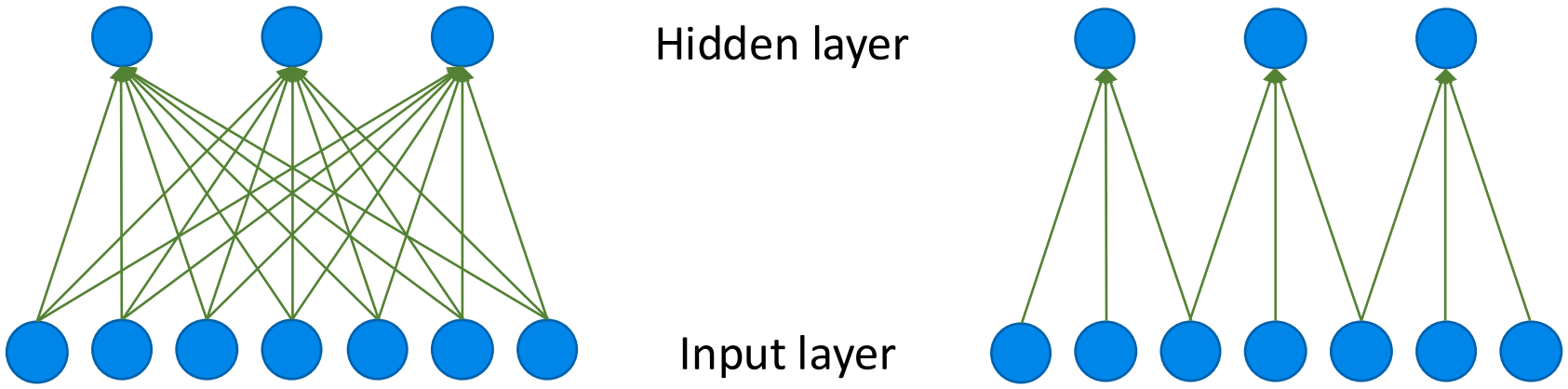
Pixels of an entity should be adequately distributed in space

Still a Dog

Spatial invariance

The semantics of an entity should not be affected much by its position

CNNs Property 1: Local Connectivity



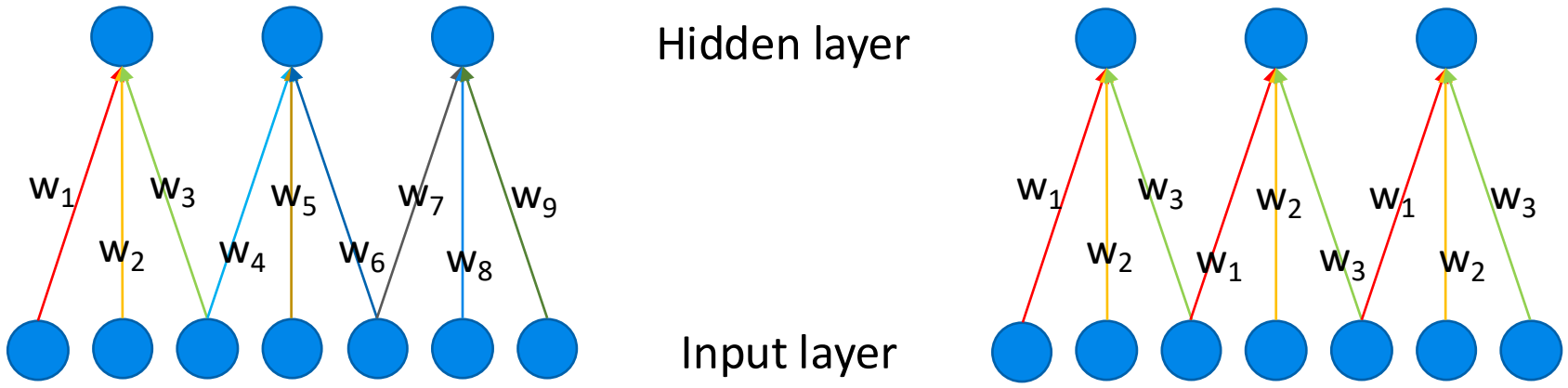
Global connectivity

Local connectivity

- # input units (neurons): 7
- # hidden units: 3
- Number of parameters
  - Global connectivity: 3 x 7 = 21
  - Local connectivity: 3 x 3 = 9

Spatial structure helps us reduce the number of connections!

CNNs Property 2: Weight Sharing



Without weight sharing

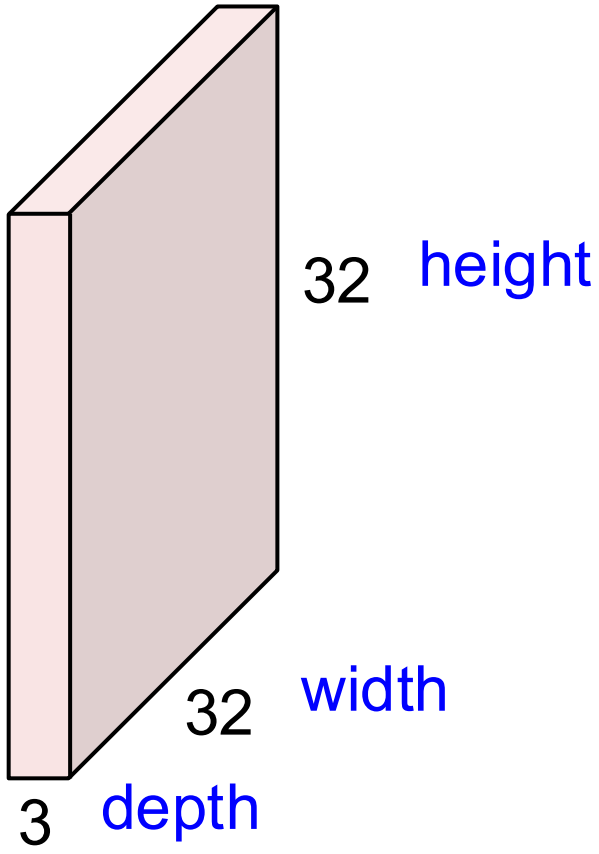
With weight sharing

- # input units (neurons): 7
- # hidden units: 3
- Number of parameters
  - Without weight sharing: 3 x 3 = 9
  - With weight sharing : 3 x 1 = 3

Spatial invariance helps us reduce number of weights (parameters to learn)!

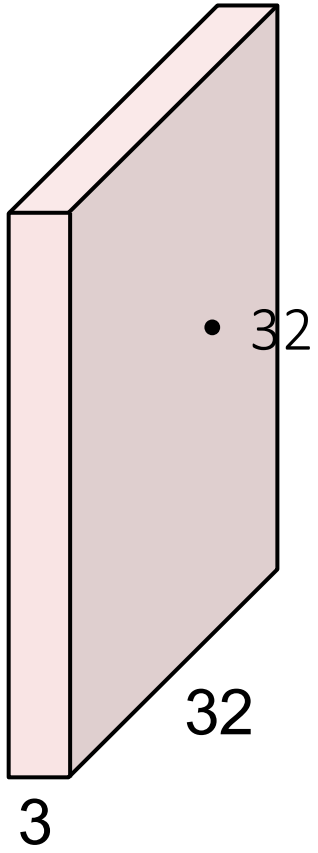
Convolution Layer

32x32x3 image -> preserve spatial structure

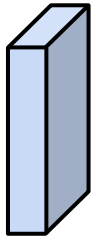


Convolution Layer

- 32x32x3 image



- 5x5x3 filter

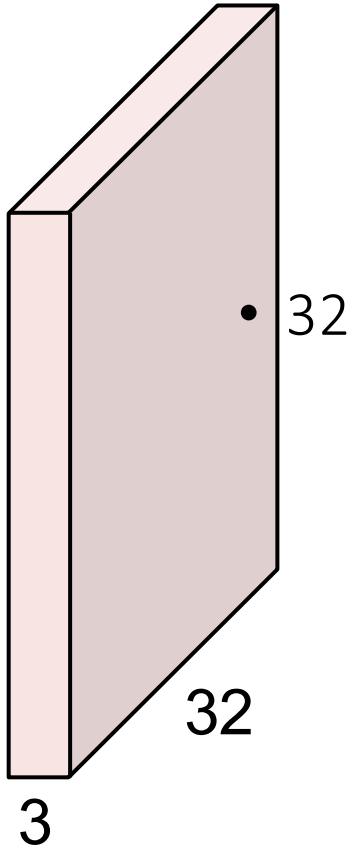


- **Convolve** the filter with the image
- i.e. “slide over the image spatially, computing dot products”

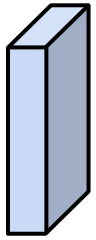
Convolution Layer

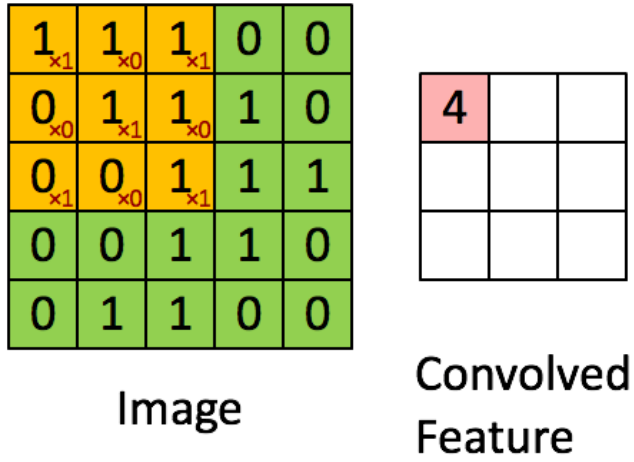
Filters always extend the full depth of the input volume

- 32x32x3 image



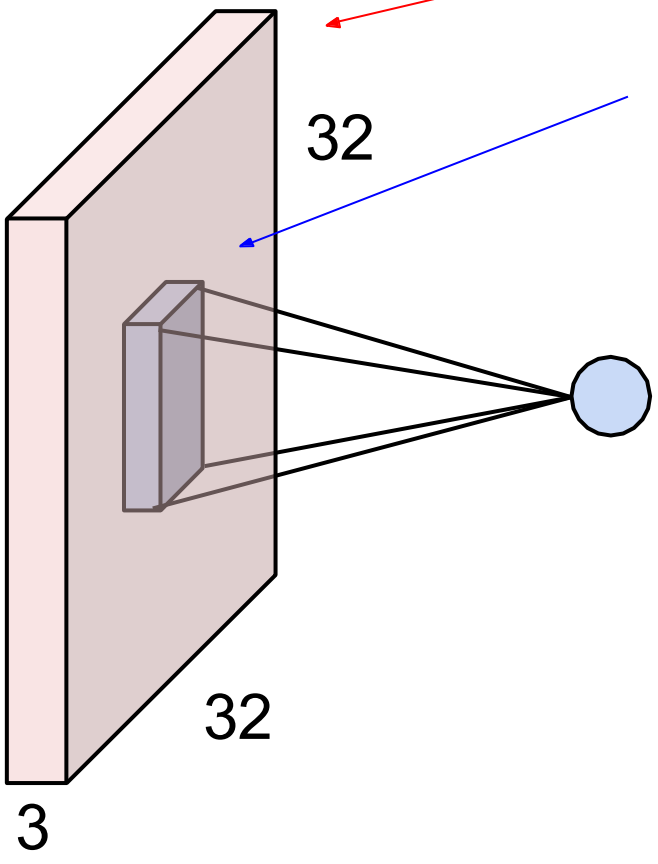
- 5x5x3 filter





- **Convolve** the filter with the image
- i.e. "slide over the image spatially, computing dot products"

Convolution Layer



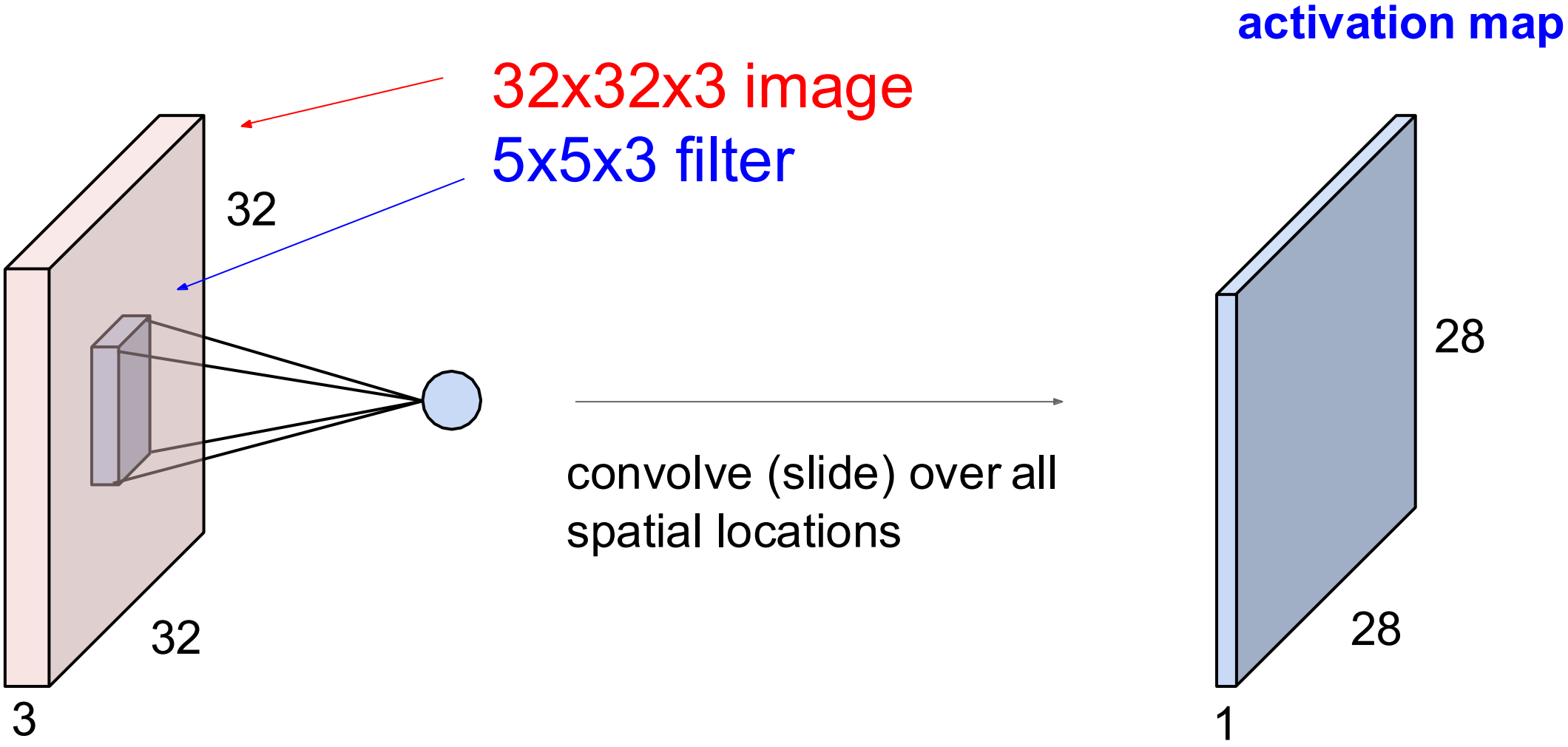
32x32x3 image

5x5x3 filter w

1 number:
the result of taking a dot product between the filter and a small 5x5x3 chunk of the image (i.e. 5*5*3 = 75-dimensional dot product + bias)

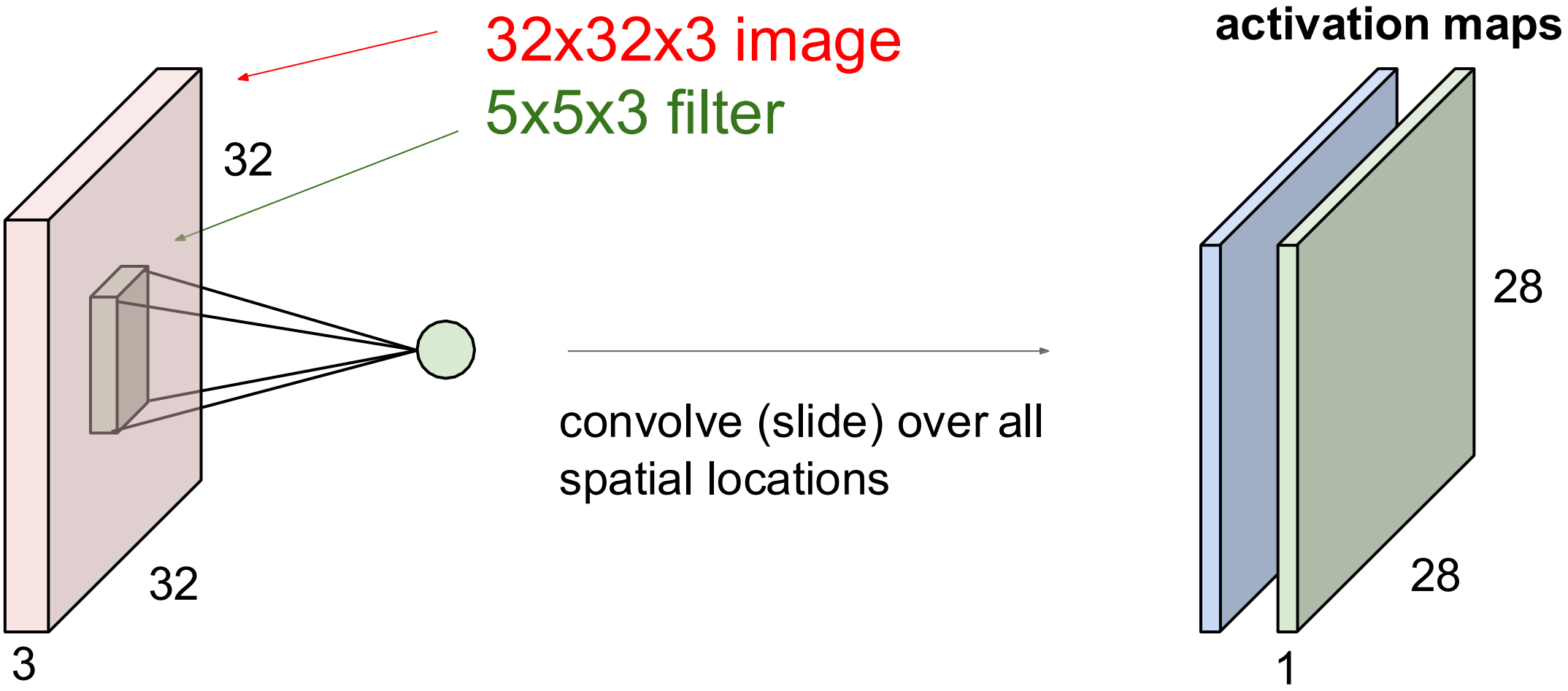
$$w^T x + b$$

Convolution Layer

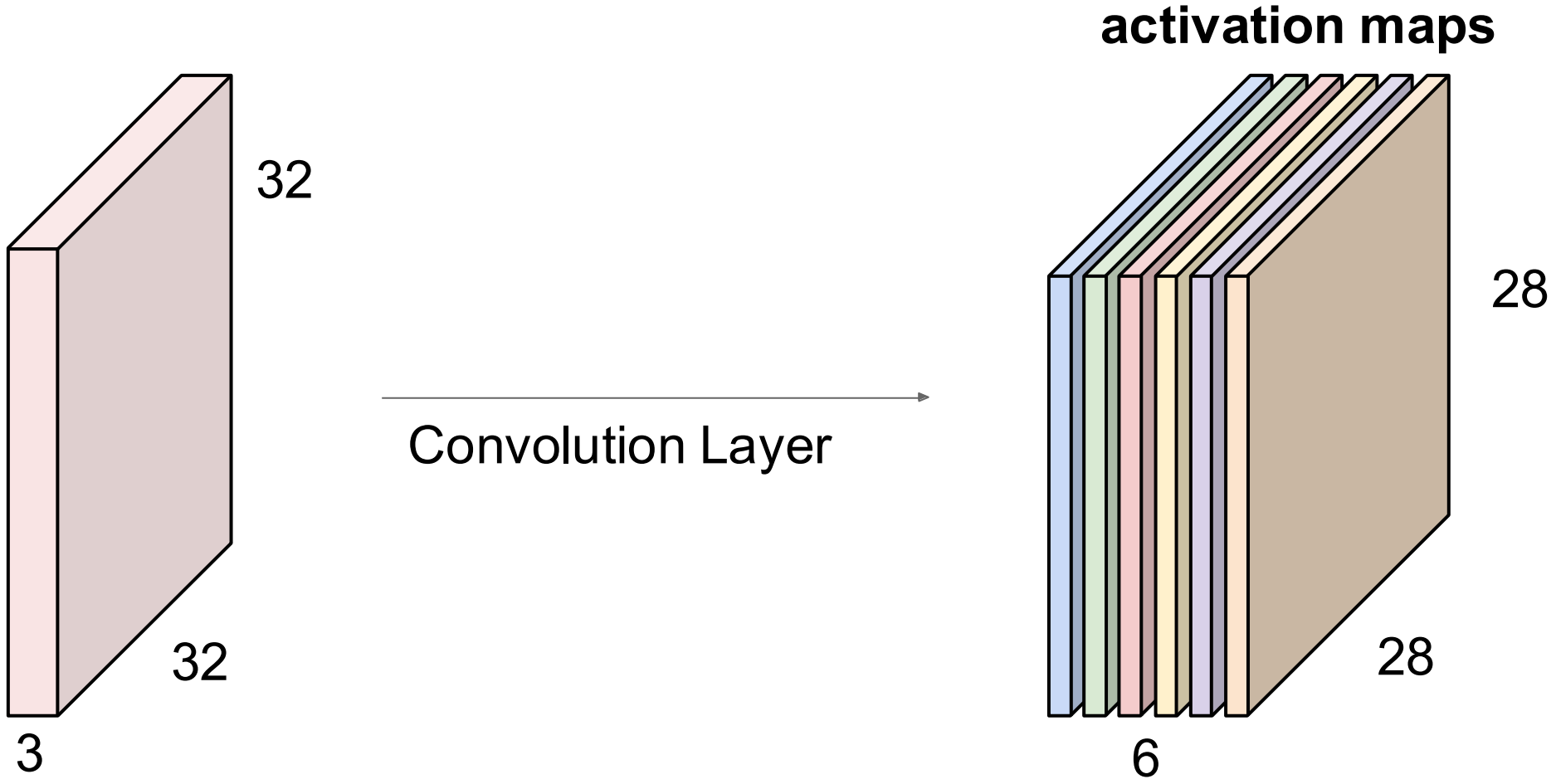


Convolution Layer

consider a second, green filter

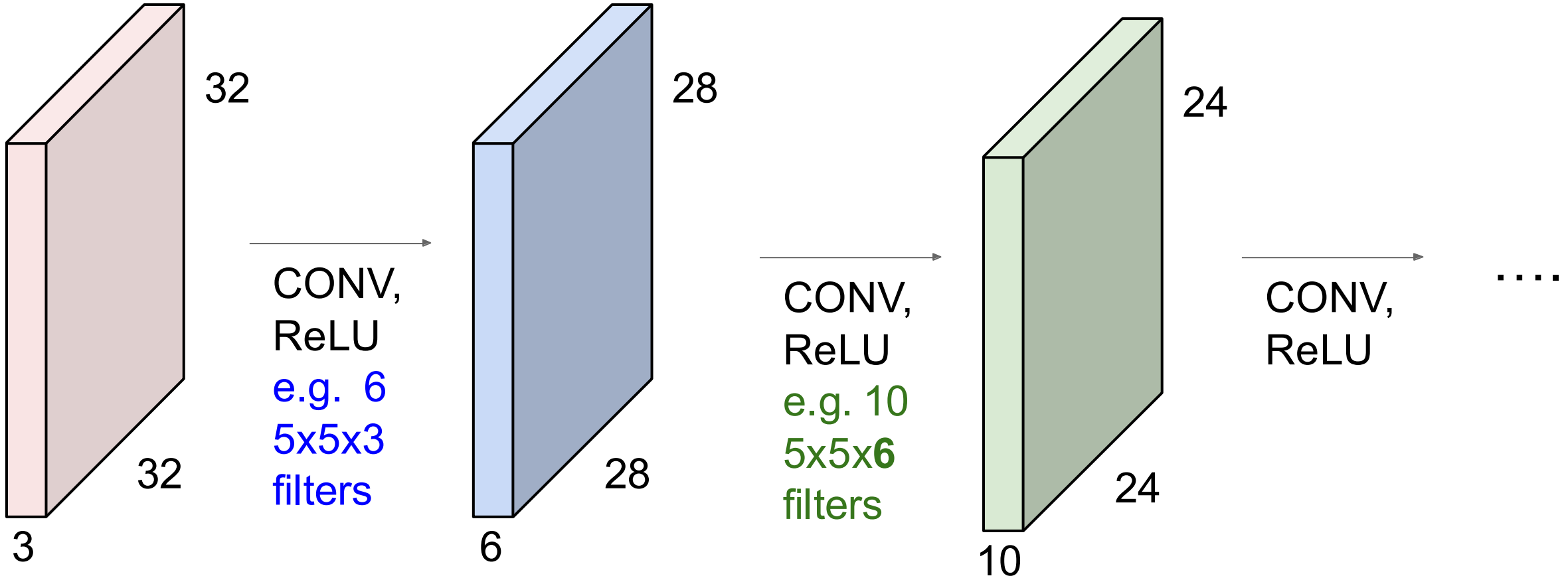


For example, if we had 6 5x5 filters, we'll get 6 separate activation maps:



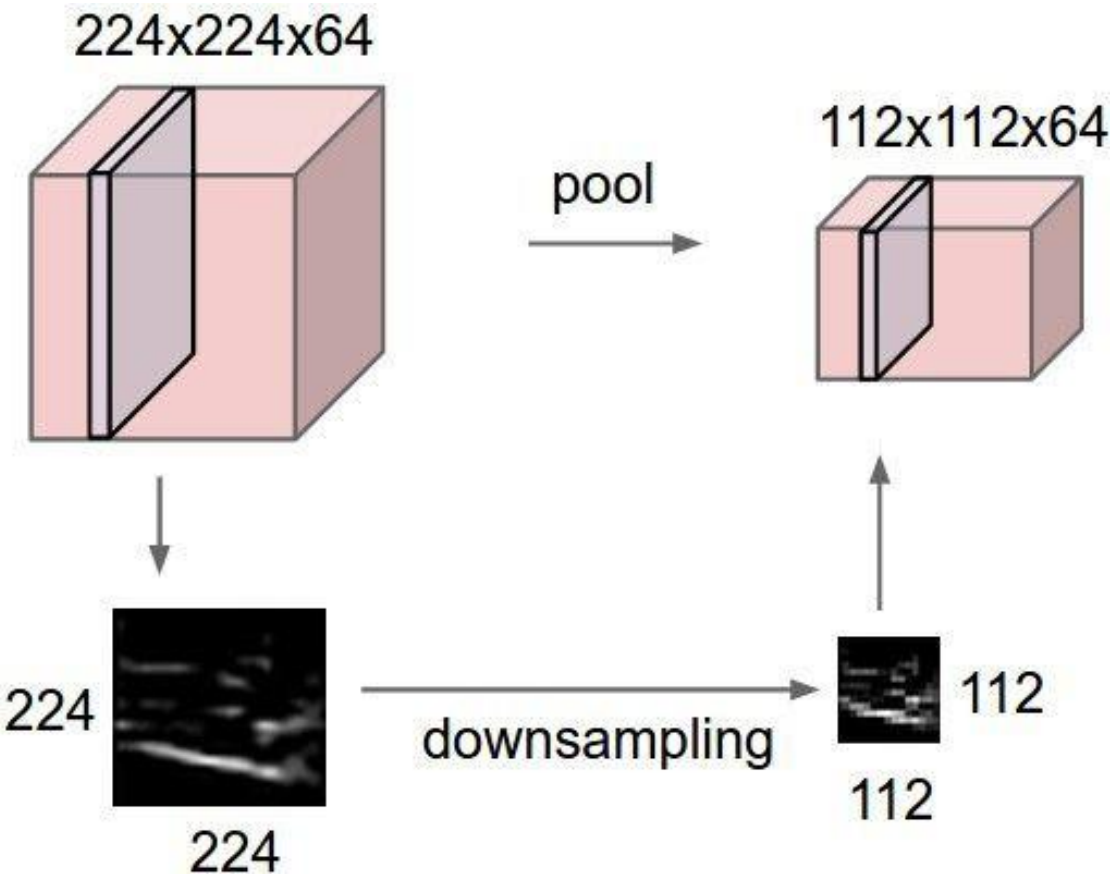
We stack these up to get a "new image" of size 28x28x6!

Preview: ConvNet is a sequence of Convolution Layers, interspersed with activation functions



Pooling layer

- makes the representations smaller and more manageable
- operates over each activation map independently:



MAX POOLING

Single depth slice

x

1	1	2	4
5	6	7	8
3	2	1	0
1	2	3	4

y

max pool with 2x2 filters
and stride 2 ()

6	8
3	4

Stride: number of pixels we're stepping over as we slide our filter across the input image

Putting them together

- Local connectivity
- Weight sharing
- Handling multiple input channels
- Handling multiple output maps

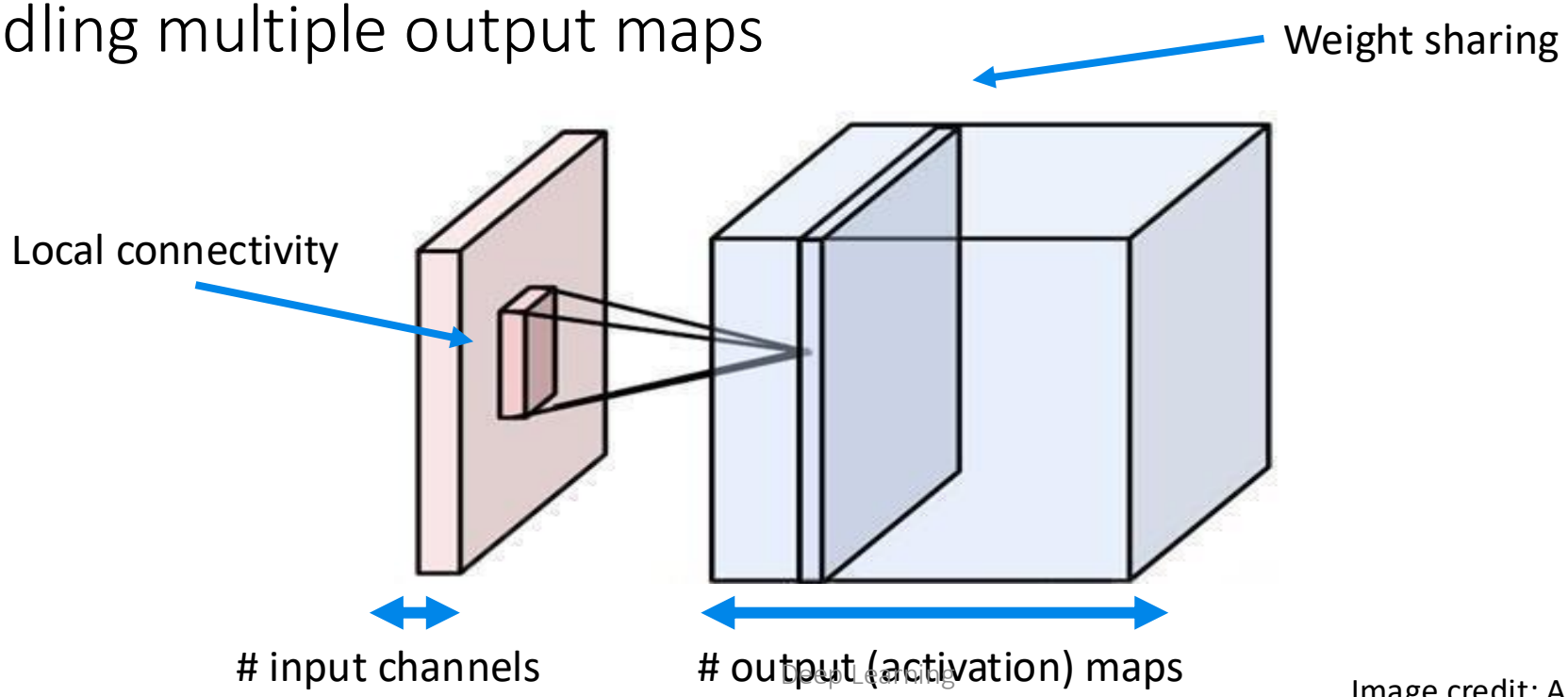
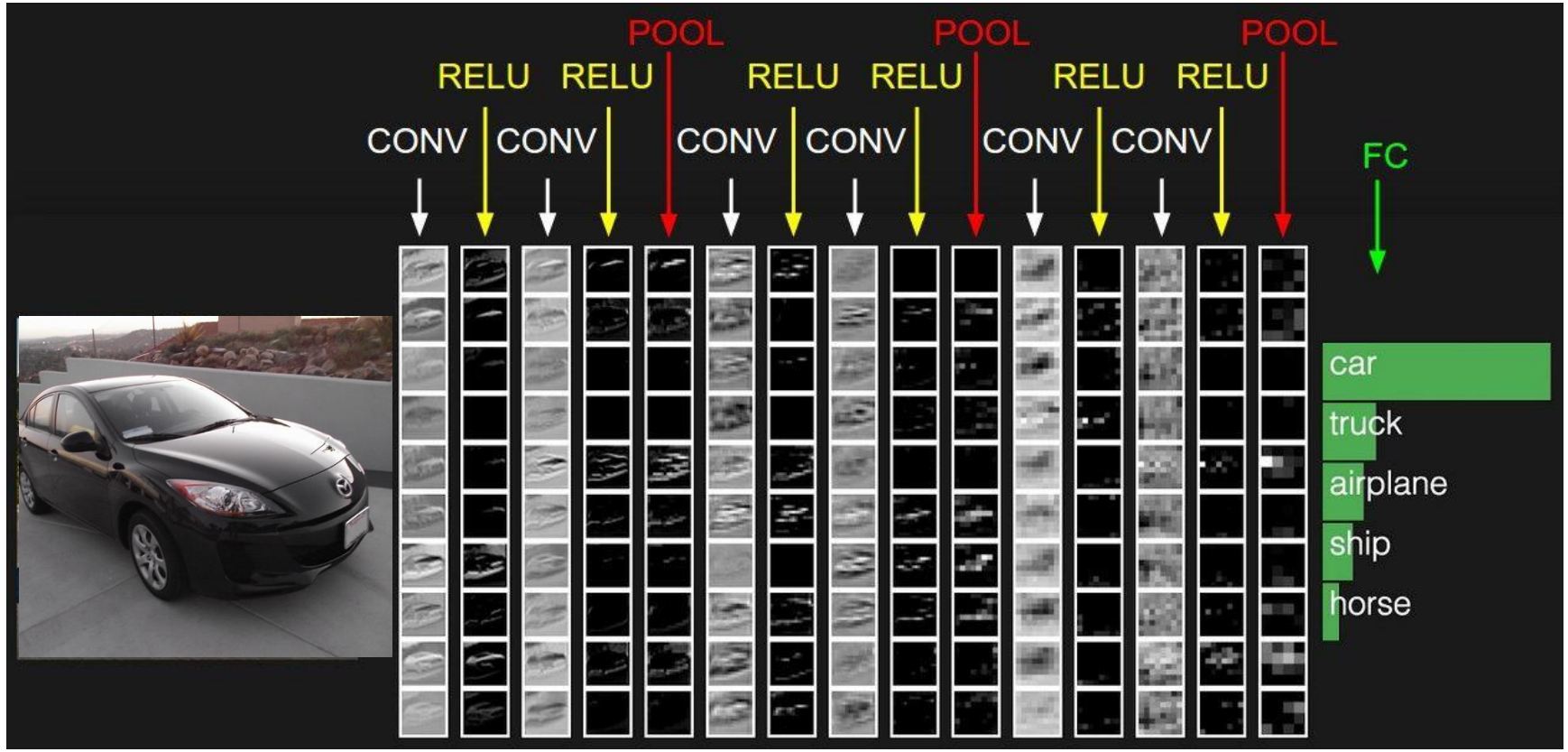


Image credit: A. Karpathy

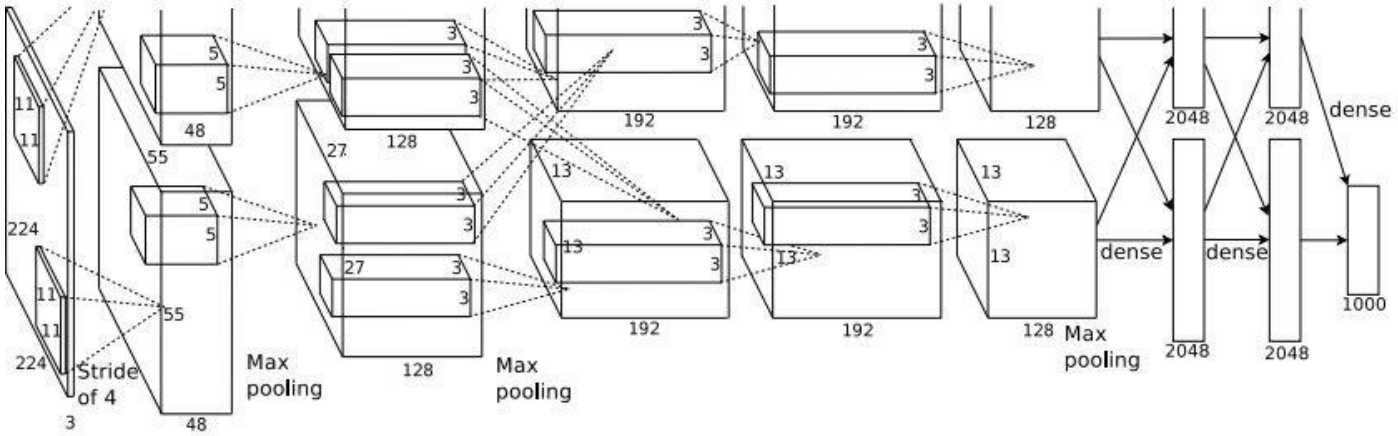
Fully Connected Layer (FC layer)

- Contains neurons that connect to the entire input volume, as in ordinary Neural Networks



Case Study: AlexNet

[Krizhevsky et al. 2012]



Full (simplified) AlexNet architecture:
[227x227x3] INPUT
[55x55x96] CONV1: 96 11x11 filters at stride 4, pad 0
[27x27x96] MAX POOL1: 3x3 filters at stride 2
[27x27x96] NORM1: Normalization layer
[27x27x256] CONV2: 256 5x5 filters at stride 1, pad 2
[13x13x256] MAX POOL2: 3x3 filters at stride 2
[13x13x256] NORM2: Normalization layer
[13x13x384] CONV3: 384 3x3 filters at stride 1, pad 1
[13x13x384] CONV4: 384 3x3 filters at stride 1, pad 1
[13x13x256] CONV5: 256 3x3 filters at stride 1, pad 1
[6x6x256] MAX POOL3: 3x3 filters at stride 2
[4096] FC6: 4096 neurons
[4096] FC7: 4096 neurons
[1000] FC8: 1000 neurons (class scores)

- First deep learning model to utilize Graphics Processing Units (GPUs) for training
- Deeper than previous neural nets
- + several other tricks, e.g., **dropout**

Figure copyright Alex Krizhevsky, Ilya Sutskever, and Geoffrey Hinton, 2012. Reproduced with permission.